



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

فاز اول پروژه

اعضای گروه

پرینا شهبواری ۴۰۱۱۷۰۶۱۵

فاطمه روستا ۴۰۱۱۰۵۹۹۱

سامیار لاجوردی ۴۰۲۱۷۱۰۵۳

نیکا قادری ۴۰۱۱۰۶۳۲۸

نام درس

تحلیل و طراحی سیستم‌ها

نیم‌سال دوم ۱۴۰۴-۱۴۰۵

نام استاد درس

جعفر حبیبی، محسن حکما

فهرست مطالب

۵	۱ فرآیند و گردش کار اجرای فاز اول
۵	۱-۱ گام اول: تحلیل سند و استخراج نیازمندی‌ها (جلسه اول)
۵	۲-۱ گام دوم: معماری کلان، فناوری‌ها و طراحی سطح بالا (جلسه دوم)
۵	۳-۱ گام سوم: تخصیص نقش‌ها و تقسیم وظایف
۶	۴-۱ گام چهارم: رفع ابهامات از طریق تعامل با ذی‌نفع (موازی با فاز طراحی)
۶	۵-۱ گام پنجم: راه‌اندازی زیرساخت مدیریت پروژه
۶	۶-۱ گام ششم: تدوین قراردادهای رابط برنامه‌نویسی (جلسه نهایی)
۷	۲ نیازمندی‌های سامانه (Agile User Stories & MoSCoW)
۷	۱-۲ Must Have
۱۰	۲-۲ Should Have
۱۱	۳-۲ Could Have
۱۲	۴-۲ Won't Have
۱۴	۳ توجیه متدولوژی: چارچوب Agile Scrum
۱۴	۱-۳ دلایل انتخاب Scrum
۱۶	۲-۳ روش‌های جایگزین و دلایل رد آن‌ها
۱۷	۳-۳ نتیجه‌گیری
۱۸	۴ ساختار تیم و نقش‌های Agile
۲۰	۵ توجیه پشته فناوری و معماری (Tech Stack & Architecture)
۲۰	۱-۵ نمودارهای مورد استفاده برای انجام معماری اولیه
۲۰	۱-۱-۵ Use Case نمودار
۲۱	۲-۱-۵ Message State Machine نمودار

۲۲		۳-۱-۵	نمودار Session State Machine
۲۲		۴-۱-۵	نمودار Register Activity Diagram
۲۴		۵-۱-۵	نمودار Group Lifecycle Sequence Diagram
۲۶		۲-۵	مقایسه رویکردهای موجود برای معماری سیستم
۲۶		۳-۵	راهکار ترکیبی (Hybrid)
۲۷		۴-۵	نمودار بسته (Package Diagram): تفکیک ماژول‌ها و لایه‌های پیاده‌سازی
۲۹		۵-۵	رویکرد Monolith-First
۳۱		۶-۵	توجیه فناوری‌ها
۳۱		۱-۶-۵	Nginx
۳۱		۲-۶-۵	ASGI Server
۳۱		۳-۶-۵	PostgreSQL
۳۲		۴-۶-۵	Celery
۳۲		۵-۶-۵	Redis
۳۳		۶-۶-۵	Docker
۳۳		۷-۵	نمودار استقرار سرویس‌ها (Service Deployment Layout)
۳۴		۸-۵	معماری اجزا (Component Architecture)
۳۶		۶	مدل‌سازی پایگاه داده (ERD)
۳۶		۱-۶	مدیریت کاربران و نمایه‌ها
۳۶		۲-۶	فضاهای کاری و گروه‌ها
۳۷		۳-۶	هسته پیام‌رسانی و ارث‌بری جداول
۳۷		۴-۶	بهینه‌سازی چت‌های خصوصی
۳۷		۵-۶	دستاوردهای کلیدی این معماری داده
۳۹		۷	طرح‌های رابط کاربری (Wireframes)

۴۰	بخش اول: قراردادهای API مدیریت هویت و نمایه (Identity & Profile Management)	۱-۸
۴۰	ثبت نام کاربر (Register User)	۱-۱-۸
۴۰	ورود کاربر (Login User)	۲-۱-۸
۴۱	خروج کاربر (Logout User)	۳-۱-۸
۴۲	درخواست بازیابی رمز عبور (Password Reset Request)	۴-۱-۸
۴۲	دریافت نمایه (Get My Profile)	۵-۱-۸
۴۳	بروزرسانی نمایه (Update My Profile)	۶-۱-۸
۴۳	مشاهده نمایه کاربری دیگران (View Another User's Profile)	۷-۱-۸
۴۵	بخش دوم: قراردادهای API فضاهای کاری (کانال‌ها، تایپک‌ها و نقش‌ها) (Workspaces)	۲-۸
۴۵	دریافت کانال‌ها (Fetch My Channels)	۱-۲-۸
۴۵	ایجاد کانال (Create a Channel)	۲-۲-۸
۴۶	ویرایش/حذف کانال (Edit/Delete Channel)	۳-۲-۸
۴۶	پیوستن به کانال (Join a Channel)	۴-۲-۸
۴۷	ایجاد نقش سفارشی (Create a Custom Role)	۵-۲-۸
۴۸	ترک کانال (Leave a Channel)	۶-۲-۸
۴۸	ویرایش/حذف نقش سفارشی (Edit/Delete Custom Role)	۷-۲-۸
۴۸	تخصیص نقش به عضو (Assign Role to Member)	۸-۲-۸
۴۹	ایجاد تایپک (Create a Topic)	۹-۲-۸
۵۰	حذف تایپک (Delete a Topic)	۱۰-۲-۸
	بخش سوم: قراردادهای API فضاهای خصوصی (گروه‌ها و چت‌های مستقیم) (Private Spaces)	۳-۸
۵۱	دریافت گروه‌ها و چت‌های مستقیم (Fetch My Groups & DMs)	۱-۳-۸
۵۱	ایجاد چت مستقیم (Create a Direct Chat)	۲-۳-۸
۵۲	حذف چت مستقیم (Delete a Direct Chat)	۳-۳-۸

۵۲	ایجاد گروه (Create a Group)	۴-۳-۸
۵۳	ویرایش اطلاعات گروه (Edit Group Details)	۵-۳-۸
۵۳	حذف یا ترک گروه (Delete or Leave a Group)	۶-۳-۸
۵۳	ارسال دعوت‌نامه گروه (Send Group Invitation)	۷-۳-۸
۵۴	پاسخ به دعوت‌نامه گروه (Respond to Group Invitation)	۸-۳-۸
۵۵	بخش چهارم: قراردادهای API هسته پیام‌رسانی (The Messaging Core)	۴-۸
۵۵	ارسال پیام متنی (Send a Message - Text)	۱-۴-۸
۵۵	پیوست رسانه به پیام (Attach Media to a Message)	۲-۴-۸
۵۶	دریافت تاریخچه پیام‌ها (Fetch Message History)	۳-۴-۸
۵۷	ویرایش پیام (Edit a Message)	۴-۴-۸
۵۷	حذف پیام (Delete a Message)	۵-۴-۸
۵۸	جستجوی متنی (Text Search)	۶-۴-۸
۵۹	بخش پنجم: قراردادهای API ویژگی‌های پیشرفته و ارتباطات بلادرنگ	۵-۸
۵۹	اتصال وب‌سوکت (چت live و اعلانات) (WebSocket Connection)	۱-۵-۸
۵۹	دریافت اعلانات از دست‌رفته (Fetch Missed Notifications)	۲-۵-۸
۶۰	زمان‌بندی پیام (Schedule a Message)	۳-۵-۸
۶۰	لغو پیام زمان‌بندی‌شده (Cancel a Scheduled Message)	۴-۵-۸

۱ فرآیند و گردش کار اجرای فاز اول

به منظور اجرای ساختاریافته و اصولی فاز اول پروژه، تیم مجموعه‌ای از جلسات هماهنگی و Brainstorming را برنامه‌ریزی و اجرا نمود. هدف از این جلسات، این بود که تمام قراردادهای فنی مربوط به نحوه پیاده‌سازی تعیین و تصویب شود. در ادامه، گام‌های طی شده در این فاز به‌طور خلاصه شرح داده شده است:

۱-۱ گام اول: تحلیل سند و استخراج نیازمندی‌ها (جلسه اول)

در نخستین جلسه، تیم به مطالعه دقیق و خط‌به‌خط سند پروژه (RFP) پرداخت. در این گام، نیازمندی‌های صریح و ضمنی سامانه استخراج شده و با استفاده از تکنیک MoSCoW اولویت‌بندی شدند. خروجی این جلسه، تدوین داستان‌های کاربری (User Stories) استاندارد بود که مبنای تمامی طراحی‌های بعدی قرار گرفت.

۲-۱ گام دوم: معماری کلان، فناوری‌ها و طراحی سطح بالا (جلسه دوم)

در جلسه دوم، تیم پیش از هر چیز بر روی معماری کلان سامانه (Macro-System Architecture) و استخوان‌بندی اصلی پروژه متمرکز شد. سپس پشته فناوری (Tech Stack) متناسبی نیز انتخاب گردید؛ شامل React برای فرانت‌اند، Django/FastAPI برای بک‌اند و RabbitMQ/WebSockets به‌عنوان پل ارتباطی سیستم‌ها. در نهایت، موجودیت‌های اصلی پایگاه داده و روابط آن‌ها در یک سطح انتزاعی و High-Level بررسی شد تا اطلاعات کافی برای شروع کار اعضا فراهم گردد، بدون آنکه تیم در مراحل اولیه درگیر جزئیات پیاده‌سازی شود.

۳-۱ گام سوم: تخصیص نقش‌ها و تقسیم وظایف

در ادامه جلسه دوم، با توجه به توانمندی‌های اعضا و بر اساس مدل مهارت‌های T-Shaped، وظایف فاز اول (طراحی ERD، طراحی Wireframe‌ها و نگارش سایر مستندات) میان اعضا تقسیم شد. همچنین چشم‌انداز نقش‌های فاز دوم (توسعه‌دهنده فرانت‌اند، برنامه‌نویس بک‌اند، مالک محصول و اسکرام مستر) به‌طور شفاف برای ورود به فاز پیاده‌سازی تعیین گردید.

۴-۱ گام چهارم: رفع ابهامات از طریق تعامل با ذی نفع (موازی با فاز طراحی)

در حین طراحی، ابهاماتی در خصوص منطق تجاری (Business Logic) سامانه به وجود آمد. مالک محصول (PO) از طریق ارتباط مستقیم با TA (به عنوان نماینده ذی نفع)، این موارد را شفاف سازی نمود. مهم ترین تصمیمات اتخاذ شده بر اساس این استعلام ها عبارتند از:

- تعیین Username به عنوان شناسه یکتای سیستم و الزامی شدن دریافت Email صرفاً برای جریان بازیابی رمز عبور.
- توافق بر سر استفاده از مکانیزم Global Delete (حذف کامل از پایگاه داده) برای گروه ها و چت های خصوصی به جای Soft Delete برای کاهش پیچیدگی.
- محدود کردن دامنه جستجو صرفاً به متن پیام ها (Text-only Search) و عدم نیاز به جستجو در نام فایل ها یا نمایه ها.
- ذخیره سازی اعلانات در پایگاه داده برای کاربرانی که آفلاین هستند، علاوه بر ارسال بلادرنگ از طریق WebSocket.

۵-۱ گام پنجم: راه اندازی زیرساخت مدیریت پروژه

پس از نهایی شدن وظایف و داستان های کاربری، بورد مدیریت پروژه در پلتفرم Jira راه اندازی شد. تمامی تسک ها در این بورد ثبت گردید تا هر یک از اعضا بتوانند روند پیشرفت کار خود و دیگران را به صورت شفاف پیگیری نمایند.

۶-۱ گام ششم: تدوین قراردادهای رابط برنامه نویسی (جلسه نهایی)

در آخرین گام از فاز اول، تیم با برگزاری یک جلسه نهایی، نیازمندی ها، داستان های کاربری و ERD طراحی شده را به قراردادهای دقیق API تبدیل کرد. خروجی این جلسه یک مستند مرجع شامل مسیرها (Endpoints)، ساختار JSON ورودی و خروجی و قوانین تجاری بود. وجود این قراردادها در فاز دوم تضمین می کند که تیم های فرانت اند و بک اند می توانند به صورت کاملاً موازی و مستقل کار کنند و از بروز تداخل در زمان یکپارچه سازی جلوگیری شود.

۲ نیازمندی‌های سامانه (Agile User Stories & MoSCoW)

با توجه به رویکرد چابک (Agile) انتخاب شده برای توسعه این پروژه و هماهنگی‌های انجام شده با ذی‌نفع (TA)، نیازمندی‌های سیستم در قالب داستان‌های کاربری (User Stories) استاندارد استخراج شده و با استفاده از تکنیک MoSCoW اولویت‌بندی گردیده‌اند. این اولویت‌بندی تضمین می‌کند که هسته اصلی پیام‌رسان در اسپرینت‌های اولیه توسعه یابد.

۱-۲ Must Have

این موارد هسته اصلی سامانه Discord-Sim را تشکیل می‌دهند. بدون تکمیل این داستان‌ها، محصول نهایی فاقد ارزش عملیاتی خواهد بود.

- **احراز هویت (Authentication):** به عنوان یک مهمان (Guest)، من می‌خواهم با استفاده از ایمیل و یک نام کاربری یکتا ثبت‌نام کنم، تا بتوانم هویت اختصاصی خود را در پلتفرم شکل دهم.

– **معیارهای پذیرش:** بررسی یکتایی Username و Email، هش کردن امن رمز عبور پیش از ذخیره، و صدور توکن‌های دسترسی بلافاصله پس از ثبت‌نام موفق.

- **احراز هویت (Authentication):** به عنوان یک کاربر (User)، من می‌خواهم با اطلاعات کاربری‌ام وارد سیستم شوم، تا بتوانم به صورت امن به چت‌های خصوصی و کانال‌های خود دسترسی داشته باشم.

– **معیارهای پذیرش:** اعتبارسنجی دقیق اطلاعات هویتی، صدور توکن‌های نشست فعال در صورت صحت، و بازگرداندن خطای 401 Unauthorized در صورت اشتباه بودن اطلاعات.

- **احراز هویت (Authentication):** به عنوان یک کاربر (User)، من می‌خواهم به صورت امن از حساب کاربری خود خارج (Log out) شوم، تا نشست فعال من پایان یابد و حسابم در دستگاه‌های اشتراکی امن بماند.

– **معیارهای پذیرش:** قرار دادن Refresh Token ارائه‌شده در Blacklist دیتابیس و ابطال کامل نشست کاربری روی سرور.

- **مدیریت نمایه (Profile Management):** به عنوان یک کاربر (User)، من می‌خواهم نمایه سایر کاربران را مشاهده کنم، تا پیش از ارسال پیام، نام نمایشی و بیوگرافی آن‌ها را ببینم.

- معیارهای پذیرش: نمایش موفق اطلاعات عمومی (نام نمایشی و بیوگرافی)؛ پنهان‌سازی تنظیمات حریم خصوصی؛ و بازگرداندن خطای 404 در صورت وجود نداشتن نام کاربری.
- **ناوبری و رابط کاربری (Navigation & UI):** به عنوان یک کاربر (User)، من می‌خواهم پس از ورود به سیستم، لیستی از تمام کانال‌ها، گروه‌ها و چت‌های مستقیم (DMs) فعال خود را مشاهده کنم، تا بتوانم به راحتی میان فضاهای کاری مختلف جابه‌جا شوم.
- معیارهای پذیرش: واکنشی صحیح تمام فضاهایی که کاربر عضو آن‌هاست؛ در چت‌های مستقیم باید شناسه کاربر مقابل پردازش (Resolve) شده و نام و آواتار وی بازگردانده شود.
- **پیام‌رسانی (Messaging):** به عنوان یک کاربر (User)، من می‌خواهم پیام‌های خود را در یک تایپیک کانال، گروه یا چت مستقیم ارسال کنم، تا بتوانم با هم‌تایان خود ارتباط برقرار کنم.
- معیارهای پذیرش: اعتبارسنجی عضویت فرستنده در فضای هدف؛ اطمینان از ارسال مقدار برای دقیقاً یک مقصد (Topic/Group/DM)؛ و ذخیره موفق رکورد در جدول پیام‌ها.
- **پیام‌رسانی (Messaging):** به عنوان یک کاربر (User)، من می‌خواهم هنگام باز کردن یک چت، تاریخچه پیام‌ها بارگذاری شود، تا بتوانم گفتگوهای پیشین را بخوانم.
- معیارهای پذیرش: واکنشی پیام‌های پیشین با استفاده از مکانیزم صفحه‌بندی (Pagination با پارامترهای Limit و Offset) جهت جلوگیری از سربار پایگاه داده.
- **پیام‌رسانی (Messaging):** به عنوان یک کاربر (User)، من می‌خواهم پیام‌های ارسال شده خود را ویرایش یا حذف کنم، تا بتوانم اشتباهات تایپی را اصلاح کرده یا اطلاعات منسوخ را پاک کنم.
- معیارهای پذیرش: بررسی تطابق درخواست‌دهنده با Sender_ID؛ در ویرایش: انتقال متن قدیم به تاریخچه و تغییر وضعیت Is_Edited؛ در حذف: اجرای Global Hard Delete.
- **پیام‌رسانی (Messaging):** به عنوان یک کاربر (User)، من می‌خواهم در متن پیام‌های گذشته جستجو کنم، تا بتوانم به سرعت بحث‌های فنی خاص را بازیابی کنم.
- معیارهای پذیرش: اجرای جستجو صرفاً روی فیلد Content؛ و اعمال محدودیت دسترسی تا کاربر تنها نتایج مربوط به فضاهایی که در آن‌ها عضو است را مشاهده کند.
- **نظارت (Moderation):** به عنوان یک مدیر کانال یا گروه (Admin)، من می‌خواهم پیام‌های نامناسب ارسال شده توسط سایر کاربران را حذف کنم، تا بتوانم فضای چت را به‌طور مؤثری نظارت کنم.

– **معیارهای پذیرش:** بررسی وجود دسترسی DELETE_MESSAGES برای درخواست دهنده؛ و اجرای حذف فیزیکی و سراسری پیام برای تمام اعضای آن فضای کاری.

- **مدیریت کانال (Channel Management):** به عنوان یک کاربر (User)، من می‌خواهم یک کانال جدید ایجاد کنم، جزئیات آن را ویرایش کنم یا آن را حذف نمایم، تا بتوانم یک فضای کاری عمومی را به‌طور کامل مدیریت کنم.

– **معیارهای پذیرش:** تخصیص خودکار نقش مالک (Owner) به سازنده؛ بررسی مجوز لازم برای ویرایش؛ و اجرای حذف آبشاری (Cascade Delete) روی تاپیک‌ها و نقش‌ها هنگام حذف کانال.

- **مدیریت کانال (Channel Management):** به عنوان یک کاربر (User)، من می‌خواهم به کانال‌های موجود بپیوندم و به تاپیک‌های آن‌ها دسترسی پیدا کنم، تا بتوانم در بحث‌های گروهی مشارکت داشته باشم.

– **معیارهای پذیرش:** ثبت رکورد کاربر در جدول مربوطه و تخصیص خودکار دسترسی‌های نقش پیش‌فرض (@everyone).

- **مدیریت کانال (Channel Management):** به عنوان یک کاربر (User)، من می‌خواهم کانالی را که پیش‌تر به آن پیوسته‌ام ترک کنم (Leave)، تا دریافت بروزرسانی‌ها برای جامعه‌ای که دیگر به آن علاقه‌ای ندارم متوقف شود.

– **معیارهای پذیرش:** حذف رکورد کاربر از دیتابیس و پاک‌سازی آبشاری تمام نقش‌های اختصاص یافته به وی در آن کانال.

- **نقش‌های کانال (Channel Roles):** به عنوان یک مدیر کانال (Channel Admin)، من می‌خواهم نقش‌های سفارشی ایجاد کنم، آن‌ها را به اعضای خاصی اختصاص دهم و دسترسی‌هایی مانند SEND_MEDIA را فعال یا غیرفعال کنم، تا بتوانم مدیریت سرور را با امنیت واگذار کرده و از اسپم جلوگیری کنم.

– **معیارهای پذیرش:** بررسی دسترسی مدیر؛ اعتبارسنجی صحت لیست مجوزها؛ و ثبت موفق اتصال بین کاربر و نقش در دیتابیس.

- **تاپیک‌های کانال (Channel Topics):** به عنوان یک مدیر کانال (Channel Admin)، من می‌خواهم تاپیک‌ها (کانال‌های متنی) را ایجاد و حذف کنم، تا بتوانم بحث‌ها را بر اساس موضوع سازمان‌دهی کنم.

– معیارهای پذیرش: بررسی دسترسی `MANAGE_TOPICS`؛ در زمان حذف، اطمینان از باقی ماندن حداقل یک تاپیک دیگر (جلوگیری از کانال بدون تاپیک)؛ و حذف آشناری پیام‌های درون تاپیک.

- چت‌های خصوصی (Private Chats): به عنوان یک کاربر (User)، من می‌خواهم یک چت مستقیم دونفره (1-on-1 DM) با کاربری دیگر آغاز کنم، تا بتوانم به صورت خصوصی ارتباط برقرار کنیم.

– معیارهای پذیرش: بررسی وجود چت قبلی بین دو کاربر؛ در صورت وجود بازگرداندن شناسه چت فعلی (200 OK) و در غیر این صورت ایجاد چت جدید (201 Created).

- مدیریت گروه (Group Management): به عنوان یک کاربر (User)، من می‌خواهم یک چت گروهی خصوصی ایجاد کنم، تا تیم من بتواند به صورت امن و بدون شلوغ کردن کانال‌های عمومی هماهنگ شود.

– معیارهای پذیرش: ثبت گروه جدید و و ست کرد کاربر به عنوان ادمین.

- مدیریت گروه (Group Management): به عنوان یک عضو گروه (Group Member)، من می‌خواهم امکان ویرایش نام گروه یا حذف کامل آن را داشته باشم، تا فضای کاری همواره مرتبط باقی بماند.

– معیارهای پذیرش: تایید عضویت درخواست‌دهنده در گروه؛ و در صورت حذف، اجرای حذف سراسری (Global Hard Delete) روی گروه و تمام پیام‌های آن.

- حریم خصوصی (Privacy): به عنوان یک کاربر (User)، من می‌خواهم دعوت‌نامه‌های گروهی را رد یا مسدود کنم، تا کنترل داشته باشم چه کسانی مرا به بحث‌های خصوصی اضافه می‌کنند.

– معیارهای پذیرش: بررسی فیلد `Allow_Group_Invitations` کاربر هدف پیش از ثبت دعوت‌نامه (بازگرداندن خطای 403 در صورت رد تنظیمات)؛ و امکان ثبت وضعیت دعوت‌نامه به عنوان DECLINED.

۲-۲ Should Have

این ویژگی‌ها ارزش محصول و قابلیت استفاده از آن را به شدت افزایش می‌دهند و در اولویت دوم توسعه قرار دارند. در صورت کمبود زمان در اسپرینت‌های پایانی، ممکن است با راهکارهای ساده‌تری جایگزین

شوند.

- **مدیریت نمایه (Profile Management):** به عنوان یک کاربر (User)، من می‌خواهم یک آواتار بارگذاری کرده و یک بیوگرافی بنویسم، تا سایر اعضا بتوانند بیشتر با پیشینه من آشنا شوند.

– **معیارهای پذیرش:** امکان آپلود تصویر با بررسی فرمت و حجم مجاز؛ امکان ثبت و ذخیره متن بیوگرافی؛ و اعمال تغییرات به گونه‌ای که بلافاصله در نمایه عمومی کاربر برای سایر اعضا قابل مشاهده باشد.

- **اشتراک‌گذاری رسانه (Media Sharing):** به عنوان یک کاربر (User)، من می‌خواهم فایل‌های رسانه‌ای (مانند عکس، ویدیو، صوت یا PDF) را به پیام‌های خود پیوست کنم، تا بتوانم محتوای مورد نظر را مستقیماً در چت به اشتراک بگذارم.

– **معیارهای پذیرش:** بررسی رعایت سقف مجاز حجم و نوع فایل ارسالی؛ تحویل موفق فایل در کنار پیام متنی؛ و امکان دانلود یا مشاهده مستقیم محتوای رسانه‌ای توسط گیرندگان پیام در چت مربوطه.

- **هویت در سرور (Server Identity):** به عنوان یک کاربر (User)، من می‌خواهم یک نام مستعار (Nickname) مختص به هر کانال تنظیم کنم، تا بتوانم بسته به بستر سرور، هویت حرفه‌ای یا غیررسمی متفاوتی داشته باشم.

– **معیارهای پذیرش:** امکان تنظیم یک نام متمایز برای هر کانال مشخص؛ نمایش این نام در پیام‌ها و لیست اعضای همان کانال؛ و استفاده از نام نمایشی پیش‌فرض کاربر در صورت عدم تنظیم نام مستعار.

۳-۲ Could Have

این داستان‌های کاربری تنها زمانی وارد بک‌لاگ اسپرینت (Sprint Backlog) می‌شوند که هسته اصلی پیام‌رسان (Must Haves) به‌طور کامل توسعه یافته و تست شده باشد.

- **بازیابی حساب کاربری (Account Recovery):** به عنوان یک مهمان (Guest)، من می‌خواهم لینک بازیابی رمز عبور را از طریق ایمیل درخواست کنم، تا در صورت فراموشی اطلاعات ورود، بتوانم مجدداً به حساب کاربری خود دسترسی پیدا کنم.

– معیارهای پذیرش: ارسال موفق ایمیل حاوی لینک زمان‌دار و یک بار مصرف در صورت وجود حساب کاربری؛ عدم نمایش خطای صریح برای ایمیل‌های ثبت‌نشده (جهت جلوگیری از نشت اطلاعات)؛ و امکان ثبت رمز عبور جدید از طریق لینک معتبر.

• **بروزرسانی‌های بلادرنگ (Real-Time Updates):** به عنوان یک کاربر (User)، من می‌خواهم پیام‌ها و اعلانات live را از طریق وب‌سوکت (WebSockets) دریافت کنم، تا برای دیدن فعالیت‌های جدید نیازی به آپدیت (Refresh) دستی صفحه نداشته باشم.

– معیارهای پذیرش: برقراری اتصال پایدار و زنده با سرور؛ و دریافت و نمایش آنی رویدادها (مانند دریافت پیام جدید و اعلانات) در رابط کاربری، بدون آنکه کاربر نیازی به Refresh دستی صفحه داشته باشد.

• **اعلانات آفلاین (Offline Notifications):** به عنوان یک کاربر (User)، من می‌خواهم سیستم، اعلانات از دست‌رفته مرا ذخیره کند، تا بتوانم پس از ورود مجدد به سیستم، دعوت‌نامه‌های گروهی یا منشن‌ها را پیگیری کنم.

– معیارهای پذیرش: تشخیص وضعیت آفلاین بودن کاربر و ذخیره رویدادهای مهم (مانند منشن‌ها و دعوت‌نامه‌ها)؛ نمایش این هشدارها به صورت یکپارچه بلافاصله پس از ورود مجدد به سیستم؛ و امکان تغییر وضعیت آن‌ها به «خوانده‌شده».

• **زمان‌بندی پیام‌ها (Message Scheduling):** به عنوان یک کاربر (User)، من می‌خواهم پیامی را برای ارسال در یک تاریخ و زمان مشخص در آینده زمان‌بندی کنم، تا بتوانم حتی زمانی که کلاینت من آفلاین است، یادآوری‌هایی را به هم‌تیمی‌هایم تحویل دهم.

– معیارهای پذیرش: امکان تعیین تاریخ و ساعت دقیق در آینده برای ارسال؛ انتشار خودکار پیام در چت مقصد در زمان مقرر، حتی در صورت خاموش یا آفلاین بودن دستگاه فرستنده؛ و امکان لغو پیام توسط فرستنده پیش از فرا رسیدن زمان موعود.

۴-۲ Won't Have

بر اساس جلسات هماهنگی با ذی‌نفع پروژه و به منظور جلوگیری از Scope Creep، پیاده‌سازی داستان‌های کاربری زیر در فاز فعلی به‌طور رسمی از دستور کار خارج شده است.

• **ارسال مجدد پیام (Message Forwarding):** به عنوان یک کاربر (User)، من می‌خواهم یک پیام را مستقیماً از یک چت به چت دیگر فوروارد کنم. (با دستور صریح TA حذف شد).

- پاسخ مستقیم (Direct Replies): به عنوان یک کاربر (User)، من می‌خواهم مستقیماً به یک پیام خاص پاسخ دهم و یک رشته بصری (Visual Thread) ایجاد کنم. (با دستور صریح TA حذف شد).
- مسدود کردن کاربر (User Blocking): به عنوان یک کاربر (User)، من می‌خواهم کاربری دیگر را از ارسال پیام مستقیم به خودم به‌طور دائم مسدود کنم. (با دستور صریح TA حذف شد).
- جستجوی پیشرفته (Advanced Search): به عنوان یک کاربر (User)، من می‌خواهم پیام‌ها را بر اساس نام کاربری یا فایل‌های پیوست جستجو کنم. (محدوده پروژه رسماً به جستجوی متنی محدود شده است).

۳ توجیه متدلوژی: چارچوب Agile Scrum

پروژه Discord-Sim با مجموعه‌ای از ویژگی‌ها مواجه است که انتخاب متدلوژی توسعه را به‌طور مستقیم تحت تأثیر قرار می‌دهند: نیازمندی‌هایی که در طول زمان ممکن است دستخوش تغییر و تکامل شوند، معماری ترکیبی با جداسازی فرانت‌اند React از بک‌اند Django/FastAPI، تیمی متشکل از چهار دانشجو با برنامه‌ی زمانی متغیر و ددلاین‌های فشرده، و فناوری‌هایی که برای اعضای تیم تازه و ناآشنا محسوب می‌شوند. با در نظر گرفتن این ویژگی‌ها، تیم ما چارچوب Agile Scrum را با اسپرینت‌های یک‌هفته‌ای به‌عنوان متدلوژی اصلی پروژه انتخاب کرده است. در ادامه، نخست دلایل انتخاب Scrum به تفصیل شرح داده می‌شود و سپس روش‌های جایگزینی که در نظر گرفته شده اما رد شده‌اند، به همراه دلایل رد آن‌ها بررسی می‌گردد.

۱-۳ دلایل انتخاب Scrum

نخستین و شاید مهم‌ترین دلیل به حلقه‌ی بازخورد Stakeholder مربوط می‌شود. در این پروژه، استاد یار (TA) عملاً نقش Stakeholder اصلی و حامی محصول (Product Sponsor) را ایفا می‌کند. به همین دلیل روش‌های مانند Waterfall که در آن صرفاً یک تحویل نهایی داریم که پروژه تمام شده در آن به کارفرما ارائه می‌شود، برای پروژه ما مناسب نیستند؛ چراکه اگر تیم برداشتی نادرست از یکی از نیازمندی‌های سند پروژه داشته باشد، این اشتباه تا زمان تحویل نهایی کشف نخواهد شد. در مقابل، Scrum این امکان را به مالک محصول (Product Owner) می‌دهد که پیشرفت تدریجی پروژه، مانند یک API بک‌اند کارکردی یا یک رابط کاربری چت فعال را به Stakeholder ارائه دهد و بازخورد بگیرد. این حلقه‌ی بازخورد پیوسته تضمین می‌کند که مسیر حرکت تیم همواره با معیارهای ارزیابی (rubric) همسو باقی بماند.

دلیل دوم مدیریت سرعت پیشرفت تیم (velocity) هست. برنامه‌ی زمانی دانشجویان به شدت متغیر است و امتحانات، سایر پروژه‌های درسی و کارهای آزمایشگاهی همگی بر زمان قابل اختصاص به پروژه اثر می‌گذارند. روش‌های سنتی معمولاً فرض می‌کنند که امکان تخصیص زمان کاری ثابت و قابل پیش‌بینی، مثلاً چهار ساعت در هفته، وجود دارد؛ فرضی که در عمل برای دانشجویان صادق نیست. Sprint Planning در Scrum به تیم اجازه می‌دهد سرعت اسپرینت (Sprint Velocity) را به صورت دینامیک تنظیم کند؛ به این معنا که اگر هفته‌ای با میان‌ترم سنگین در پیش باشد، تیم می‌تواند برای همان اسپرینت یک هفته‌ای تعداد کمتری Story Points متعهد شود، بدون آنکه برنامه‌ی کلان پروژه مختل شود.

دلیل سوم به منحنی یادگیری فناوری و کنترل تجربی فرایند (Empirical Process Control) مربوط می‌شود. معماری ترکیبی این پروژه از فناوری‌های پیشرفته و متنوعی از جمله React، WebSocket

RabbitMQ، FastAPI و Django استفاده می‌کند. روش‌هایی مثل Waterfall فرض می‌کنند که همواره می‌توان قبل از نوشتن کد، یک تخمین زمانی دقیق برای تسک مربوطه ارائه داد؛ در حالی که عملاً غیرممکن است بدون تجربه‌ی قبلی، زمان دقیق لازم برای راه‌اندازی مثلاً یک Consumer خاص را در RabbitMQ تخمین زد. Scrum بر پایه‌ی اصل تجربه‌گرایی (empiricism)، یعنی یادگیری از طریق عمل، بنا شده است؛ اگر یک وظیفه‌ی فنی بیش از زمان پیش‌بینی شده طول بکشد، تیم در پایان همان اسپرینت یک‌هفته‌ای با آن سازگار می‌شود. باید توجه داشت که مهارت فنی تیم در طول زمان رشد می‌کند و برنامه باید بتواند خود را با این رشد هماهنگ کند.

دلیل چهارم به محافظت از هسته‌ی پروژه و مدیریت دامنه (Scope Management) مربوط می‌شود. پروژه دارای ویژگی‌های اجباری مانند پیام‌رسانی و ویژگی‌های جانبی مانند پیام‌های زمان‌بندی شده است. در رویکردهایی مثل Waterfall، اگر زمان پروژه به پایان برسد، و به هر دلیلی محصول نهایی آماده نشده باشد، معمولاً سیستمی به دست می‌آید که هشتاد درصد آن کدنویسی شده اما فاز اجرایی آن هنوز انجام نشده و نمی‌توان از لاجیک نوشته شده استفاده کرد. به طور مثال سیستمی را در نظر بگیرید که بخش فرانت و بک آن پیاده شده‌اند اما هنوز ارتباط بین آن‌ها برقرار نشده است. همچنین پروژه‌ای عملاً هیچ ارزش اجرایی نخواهد داشت زیرا فاز یکپارچه‌سازی به انتهای کار موکول شده است. در مقابل، Scrum تیم را به ساخت عمودی (vertical) محصول وادار می‌کند. با اولویت‌بندی داستان‌های کاربری ضروری (Must-Have User Stories) در اسپرینت‌های ابتدایی، تیم تضمین می‌کند که بعد از گذشت مثلاً نیمی از مهلت کل پروژه، یک نسخه‌ی کارکردی، هرچند ابتدایی، از کل سیستم وجود داشته باشد. در صورت کمبود زمان، ویژگی‌های جانبی حذف می‌شوند اما هسته‌ی اصلی پروژه سالم و قابل ارائه باقی می‌ماند.

دلیل پنجم و آخر در این بخش، (که مرتبط با دلیل قبلی هم هست) جلوگیری از Big Bang Integration از طریق یکپارچه‌سازی زودهنگام است. جداسازی تیم‌های فرانت‌اند و بک‌اند ریسک ایجاد پایگاه‌های کد جزیره‌ای و ناهماهنگ را به همراه دارد. در رویکردهایی مثل Waterfall، تیم‌های فرانت‌اند و بک‌اند معمولاً برای ماه‌ها به صورت مجزا و بدون ارتباط کار می‌کنند که در نهایت به وضعیتی موسوم به Integration Hell می‌انجامد؛ یعنی زمانی که تلاش برای اتصال رابط کاربری به API با مشکلات گسترده و پیش‌بینی نشده مواجه می‌شود. با اجرای دقیق تعریف اتمام کار (Definition of Done) در پایان هر اسپرینت یک‌هفته‌ای، کد React و کد Django ملزم به یکپارچه‌سازی و آزمایش پیوسته با یکدیگر هستند و در نتیجه باگ‌ها در مراحل ابتدایی، در حالی که هنوز کوچک و قابل رفع‌اند، در قالب تست‌کیس‌های Functional, Perfor- mance, etc شناسایی می‌شوند.

۲-۳ روش‌های جایگزین و دلایل رد آنها

علاوه بر Waterfall که در دلیل‌های فوق به‌طور مستقیم با Scrum مقایسه شد، دو چارچوب Agile و یک مدل iterative دیگر نیز برای این پروژه بررسی و در نهایت کنار گذاشته شدند.

نخستین گزینه‌ی جایگزین، Kanban است که برخلاف Scrum، به‌جای اسپرینت‌های زمان‌بندی‌شده، بر اساس یک جریان پیوسته (continuous flow) از وظایف عمل می‌کند که به‌طور مستمر از حالت To Do به Done منتقل می‌شوند. Kanban برای تیم‌های نگهداری یا پشتیبانی فنی بسیار مناسب است، اما فقدان ددلاین‌های Strict اسپرینت در آن، در محیط دانشگاهی به‌سادگی می‌تواند زمینه‌ساز به‌تعویق افتادن کارها شود. در مقابل، چارچوب‌بندی زمانی Strict و یک‌هفته‌ای Scrum باعث می‌شود نظم حفظ شود و برنامه عقب نیفتد.

گزینه‌ی دوم، مدل Spiral است که یک مدل iterative با تأکید شدید بر تحلیل ریسک و ساخت نمونه‌های اولیه‌ی دورریز (throwaway prototypes) پیش از ساخت محصول نهایی است. این مدل برای پروژه‌های بزرگ و پرریسک، مانند نرم‌افزارهای حوزه‌ی هوافضا طراحی شده است. برای یک شبیه‌سازی Discord، حوزه‌ی مسئله به‌خوبی شناخته‌شده است (نیاز چندانی به استفاده از پروتوتایپ‌های مختلف ندارد) و صرف هفته‌ها برای تحلیل ریسک‌های مثلاً ارسال یک پیام متنی، صرفاً سربار غیرضروری ایجاد می‌کند و زمانی که می‌شد برای کدنویسی صرف کرد را هدر می‌دهد. Scrum در مقابل، چارچوبی به‌مراتب سبک‌تر است؛ این چارچوب وجود ریسک‌هایی مانند یادگیری WebSocket را می‌پذیرد، اما این ریسک‌ها را از طریق اجرای سریع و بازخورد تجربی مدیریت می‌کند، نه از طریق جلسات تحلیل ریسک نظری و طولانی.

گزینه‌ی سوم، روش Extreme Programming (XP) است که مجموعه‌ای از قواعد مهندسی سخت‌گیرانه از جمله توسعه‌ی مبتنی بر تست (Test-Driven Development) و برنامه‌نویسی زوجی (Pair Programming) به‌صورت اجباری تعیین می‌کند.

با توجه به اینکه تیم با یک پشته‌ی فناوری متنوع شامل React، Django/FastAPI و RabbitMQ کار می‌کند، ملزم کردن تیم به نوشتن تست‌های کامل پیش از درک کامل چارچوب‌های زیربنایی، عملاً روند توسعه را فلج می‌کند. Scrum در مقابل تنها قواعد مدیریت پروژه را تعیین می‌کند. (نه قواعد مهندسی) (را) این چارچوب به توسعه‌دهندگان اجازه می‌دهد ابتدا یک راه‌حل تقریبی برای درک یک فناوری جدید بسازند و سپس آن را بازآرایی (refactor) کنند، بدون آنکه اصول متدولوژی نقض شود.

۳-۳ نتیجه گیری

در نهایت ، انتخاب چارچوب Scrum برای توسعه‌ی پروژه Discord-Sim یک تصمیم استراتژیک و متناسب با واقعیت‌های این پروژه دانشگاهی است. این متدولوژی با ایجاد تعادلی دقیق میان انعطاف‌پذیری و ساختاریافتگی، به تیم این امکان را می‌دهد که پیچیدگی‌های مربوط به پیاده‌سازی و چالش‌های یادگیری فناوری‌های جدید را به‌طور مؤثری مدیریت کند. در حالی که متدولوژی‌های جایگزین نظیر Waterfall، Kanban، Spiral و XP هر یک نقاط قوت خود را دارند، اما هیچ‌کدام نتوانستند نیازهای اساسی این تیم از جمله دریافت بازخورد مستمر از Stakeholder، مدیریت ددلاین‌های فشرده، و جلوگیری از بحران‌های Integration نهایی را همزمان برآورده سازند. با تکیه بر اسپرینت‌های یک‌هفته‌ای و رویکرد توسعه تدریجی، Scrum نه تنها مسیر پیشرفت تیم را با معیارهای ارزیابی پروژه همسو نگه می‌دارد، بلکه تضمین می‌کند که در نهایت یک هسته نرم‌افزاری پایدار، کارآمد و قابل توسعه در زمان مقرر ارائه خواهد شد.

۴ ساختار تیم و نقش‌های Agile

ساختار تیم در پروژه Discord-Sim بر اساس فلسفه مهارت‌های T-Shaped بنا شده است. در یک تیم دانشجویی چهار نفره، ایجاد سیلوهای فنی ایزوله که در آن هر فرد تنها به یک بخش خاص محدود شود، ریسک بسیار بالایی دارد؛ زیرا در صورت عدم حضور یکی از اعضا، کل فرایند توسعه متوقف می‌شود. در مدل T-Shaped، هر یک از اعضا دارای یک تخصص عمیق و مسئولیت مشخص در یک حوزه (بخش عمودی حرف T) هستند، اما در عین حال دانشی از سایر بخش‌های پشته فناوری (Tech Stack) نیز دارند (بخش افقی حرف T) تا بتوانند در مواقع لزوم به یکدیگر کمک کرده و از ایجاد گلوگاه‌ها جلوگیری کنند. همچنین تیم از مدل Working Scrum Master و Working Product Owner استفاده می‌کند تا هیچ منبعی در تیم محدود به کارهای صرفاً مدیریتی نشده و بالاترین بهره‌وری حاصل شود.

تخصیص نقش‌ها و مسئولیت‌ها به شرح زیر است:

فاطمه روستا: عضوی از تیم توسعه (Development Team) است که در کنار برنامه‌نویسی بخش‌های تعاملی رابط کاربری با چارچوب React، مسئولیت طراحی نمودار موجودیت-رابطه (ERD) را نیز بر عهده دارد. واگذاری طراحی ERD به توسعه‌دهنده فرانت‌اند بر اساس رویکرد مدرن UI-Driven Development انجام شده است. این تصمیم تضمین می‌کند که پایگاه داده دقیقاً برای پاسخگویی به نیازهای واقعی رابط کاربری طراحی می‌شود، نه به عنوان یک تئوری انتزاعی در سمت سرور. این امر باعث شکل‌گیری ساختار اطلاعاتی بهینه و جلوگیری از پیچیدگی‌های غیرضروری و Overkill می‌گردد.

پرنیا شهسواری: دیگر عضو تیم توسعه (Development Team) است که همگام با توسعه و کدنویسی اجزای فرانت‌اند، وظیفه طراحی Wireframe‌ها را بر عهده گرفته است. مسئولیت وی در این فاز، مشخص کردن ساختار بصری و جریان اطلاعات در صفحات مختلف پلتفرم است. این ساختارها مستقیماً به عنوان نقشه راه برای توسعه یکپارچه فرانت‌اند و همچنین راهنمای مدل‌سازی داده‌ها برای طراحی دقیق‌تر ERD عمل کرده و پیوستگی میان طراحی و اجرا را تضمین می‌کنند.

سامیار لاجوردی: نقش Product Owner و مشارکت در معماری بک‌اند را بر عهده دارد. مسئولیت وی در بخش مدیریتی شامل تنظیم منطق تجاری (Business Logic)، تعریف نیازمندی‌ها و جلوگیری از گسترش بی‌رویه دامنه پروژه (Scope Creep) است. علاوه بر این، مسئولیت تدوین گزارش معماری کلان سیستم و نگارش توجیه فنی پشته فناوری (Tech Stack) نیز به وی واگذار شده بود. (مربوط به فاز اول) تخصص وی در توسعه بک‌اند، بینش فنی لازم را برای ارزیابی امکان‌سنجی ویژگی‌ها و تدوین User Stories واقع‌بینانه فراهم می‌سازد.

نیکا قادری: به عنوان توسعه‌دهنده بک‌اند در پیاده‌سازی منطق سرور مشارکت دارد و همزمان مسئولیت

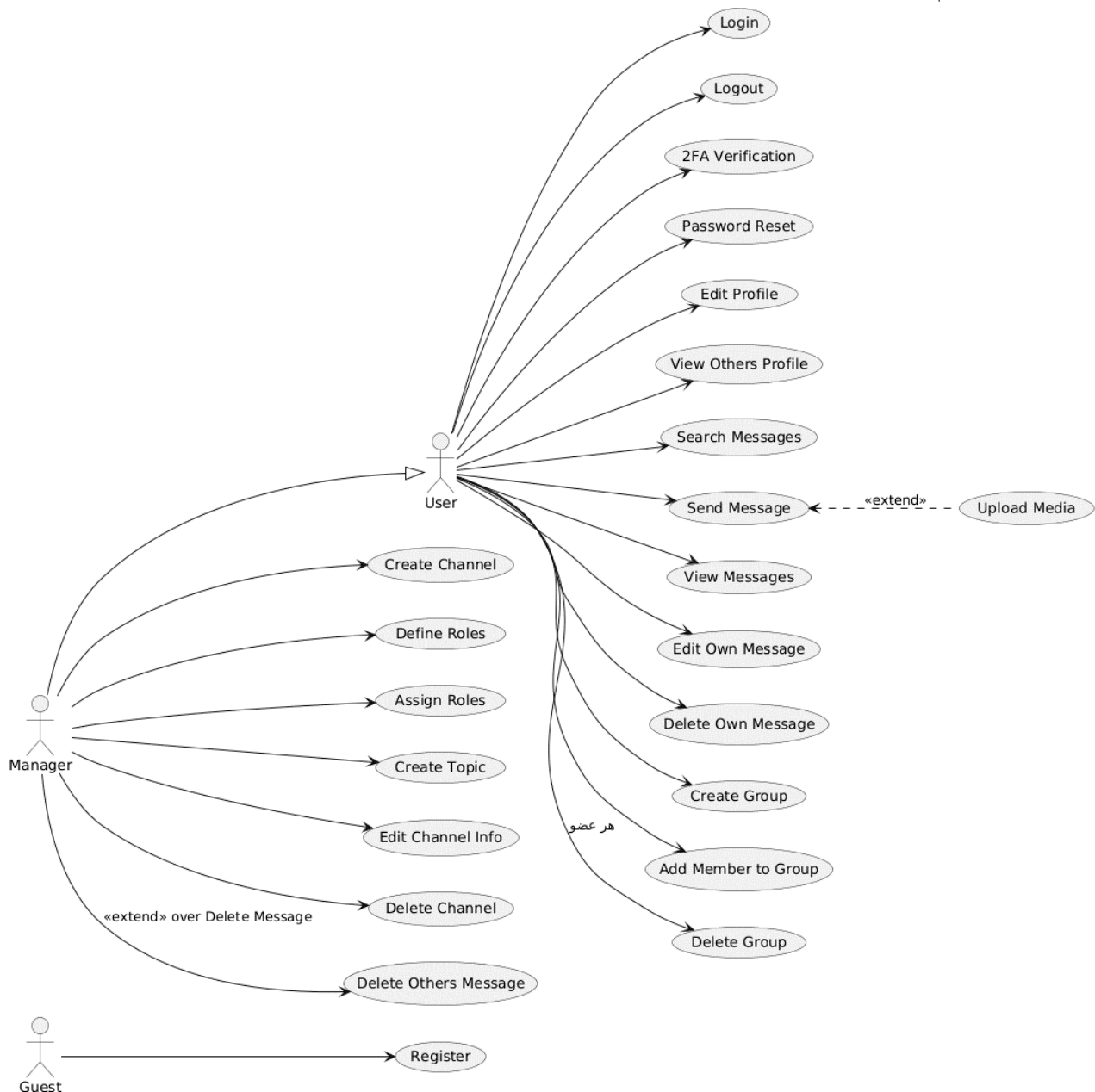
Scrum Master و تدوین قراردادهای رابط برنامه‌نویسی (API Contracts) را بر عهده گرفته است. شناخت وی از توابع بک‌اند و نحوه پردازش داده‌ها کمک می‌کند تا قراردادهای ارتباطی به‌دقت تنظیم شوند. وی به‌عنوان یک Working Scrum Master، علاوه بر کدنویسی هسته سیستم، مسئولیت نگارش توجیه متدلوژی پروژه (Methodology Justification) و گزارش تبیین نقش‌های ساختاری اعضای تیم را بر عهده داشته (فاز اول) و وظیفه شناسایی موانع و تسهیل ارتباط میان تیم‌های فرانت‌اند و بک‌اند را ایفا می‌کند.

۵ توجیه پشته فناوری و معماری (Tech Stack & Architecture)

۱-۵ نمودارهای مورد استفاده برای انجام معماری اولیه

۱-۱-۵ Use Case نمودار

از این نمودار برای ترسیم مرز سیستم، شناسایی نقش‌ها (کاربر معمولی، مدیر، مهمان) و نمایش خدمات اصلی که سیستم باید ارائه دهد، بدون ورود به جزئیات فنی استفاده می‌شود تا کمک کند بفهمیم دقیقاً چه کاری باید انجام شود.

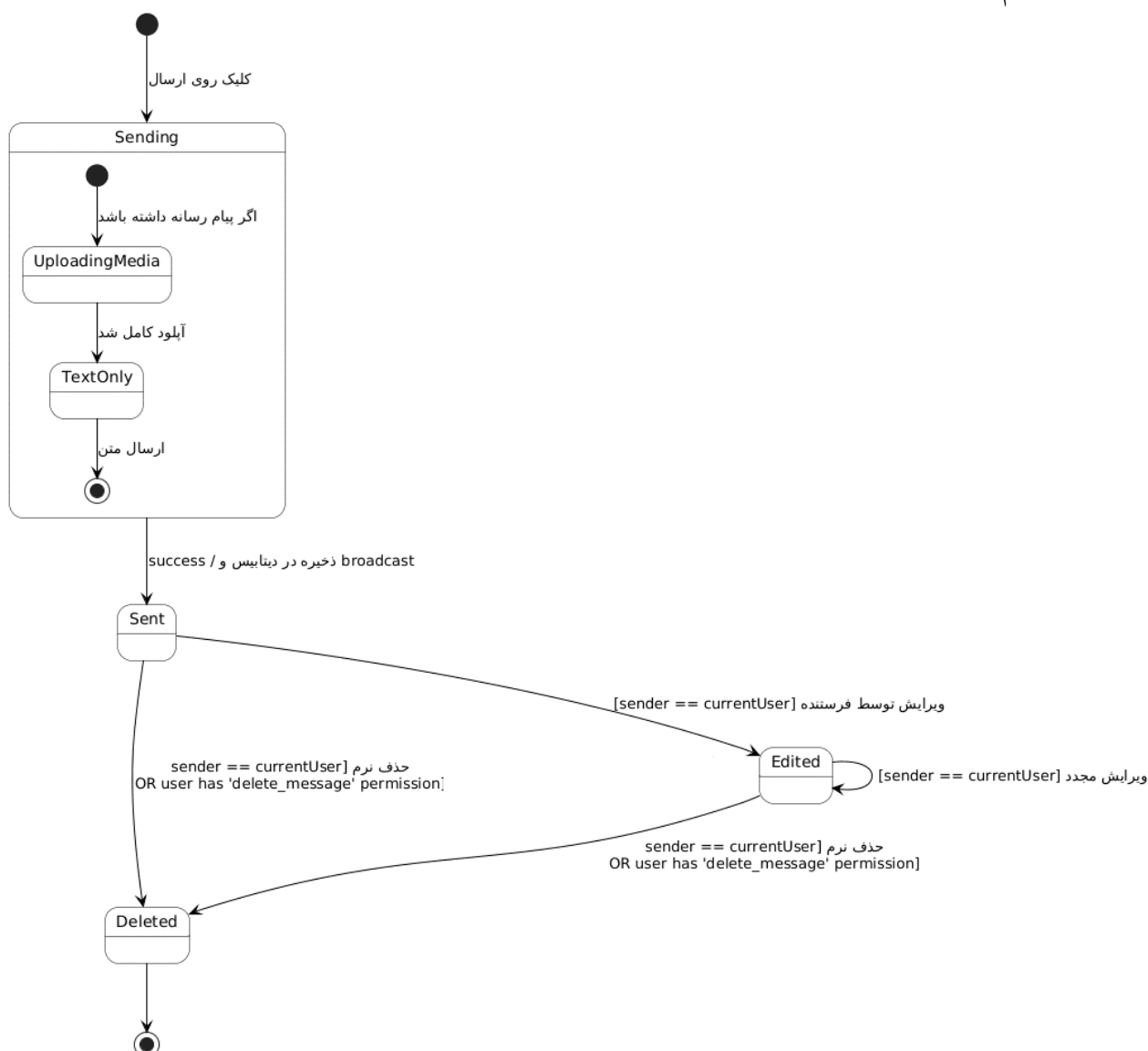


شکل ۱: نمودار Use Case

نتیجه: چون همهٔ موارد کاربرد در یک جعبهٔ «سیستم» قرار گرفتند و میان آن‌ها مرز مجزایی ترسیم نشد، مدیر صرفاً یک نوع کاربر تخصصی‌تر است، نه یک سیستم جداگانه. این یعنی می‌توان منطق کانال، گروه و پیام را در یک بدنهٔ نرم‌افزاری واحد تعریف کرد.

۲-۱-۵ نمودار Message State Machine

پیام‌ها دارای رفتاری پویا هستند؛ ارسال می‌شوند و ممکن است ویرایش یا حذف گردند. با استفاده از این نوع نمودار و نشان دادن وضعیت‌ها (Deleted، Edited، Sent) و رویدادهای تغییر، می‌توان طراحی دقیق‌تری انجام داد.



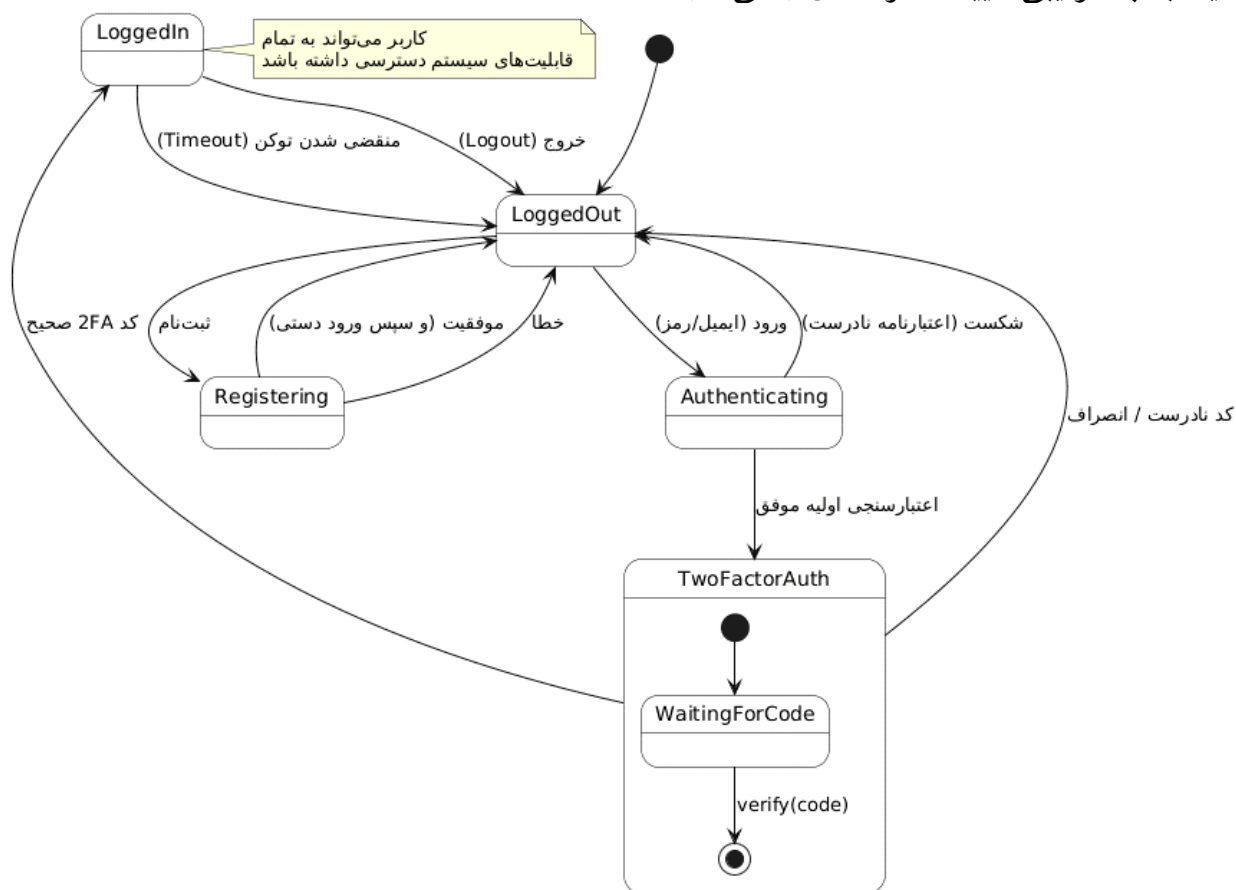
شکل ۲: نمودار Message State Machine

نتیجه: تمام تغییر وضعیت‌های پیام در یک ماشین حالت واحد رخ می‌دهند که توسط یک ماژول داخلی مدیریت می‌شود. هیچ اشاره‌ای به ارسال رویداد به سرویس خارجی یا هماهنگی توزیع‌شده برای تغییر

«حذف» یا «ویرایش» دیده نمی‌شود؛ این منطق به صورت متمرکز در همان بستر یکپارچه قابل اجراست.

۳-۱-۵ نمودار Session State Machine

برای شفاف‌سازی وضعیت‌های احراز هویت و انتقال‌های بین آن‌ها، از یک نمودار State Machine دیگر نیز استفاده شد. چون نیازمندی‌ها شامل 2FA اجباری بود، باید دقیقاً مشخص می‌شد که کاربر بعد از ورود اولیه، به چه ترتیبی تأیید دومرحله‌ای را می‌گذراند.

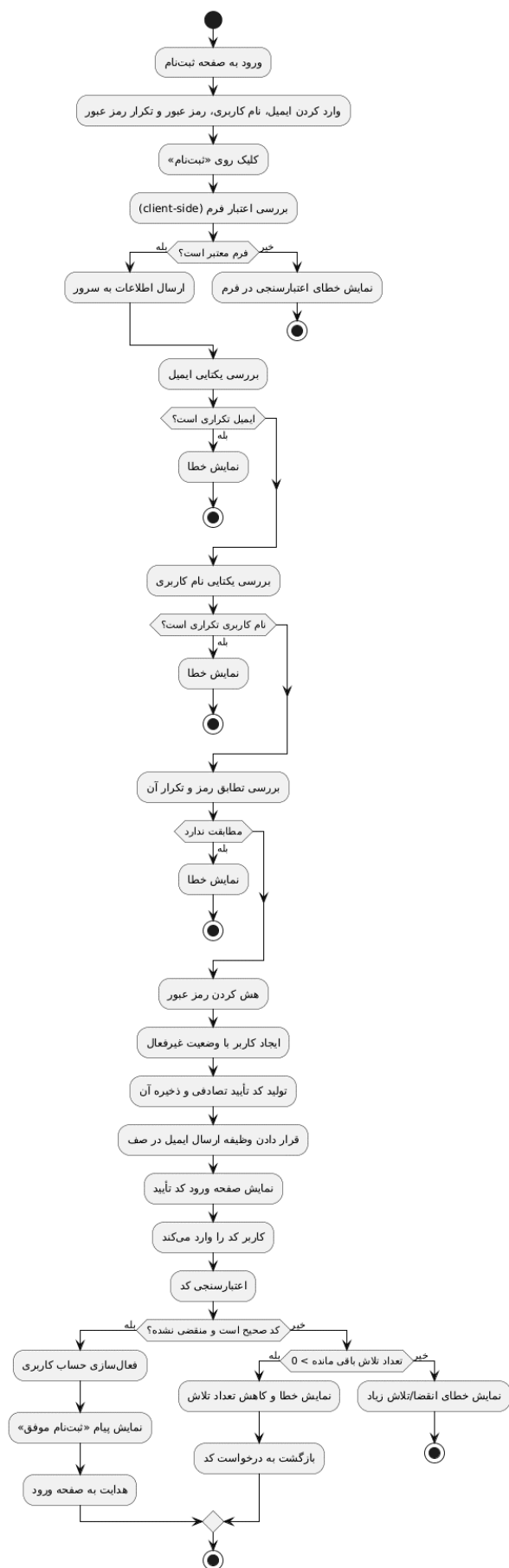


شکل ۳: نمودار Session State Machine

نتیجه: حالت‌های «Authenticating»، «TwoFactorAuth» و «LoggedIn» در یک جریان (Flow) و زیر یک چتر مدیریت می‌شوند. این یعنی احراز هویت و نشست‌ها جزئی از هسته اصلی برنامه هستند و توسط همان سرور و پایگاه داده (PostgreSQL/Redis) پیگیری می‌شوند، نه یک سرویس هویت مجزا.

۴-۱-۵ نمودار Register Activity Diagram

ثبت نام، جریانی با تصمیم‌های متعدد بود که بهتر در یک Activity Diagram دیده می‌شد.



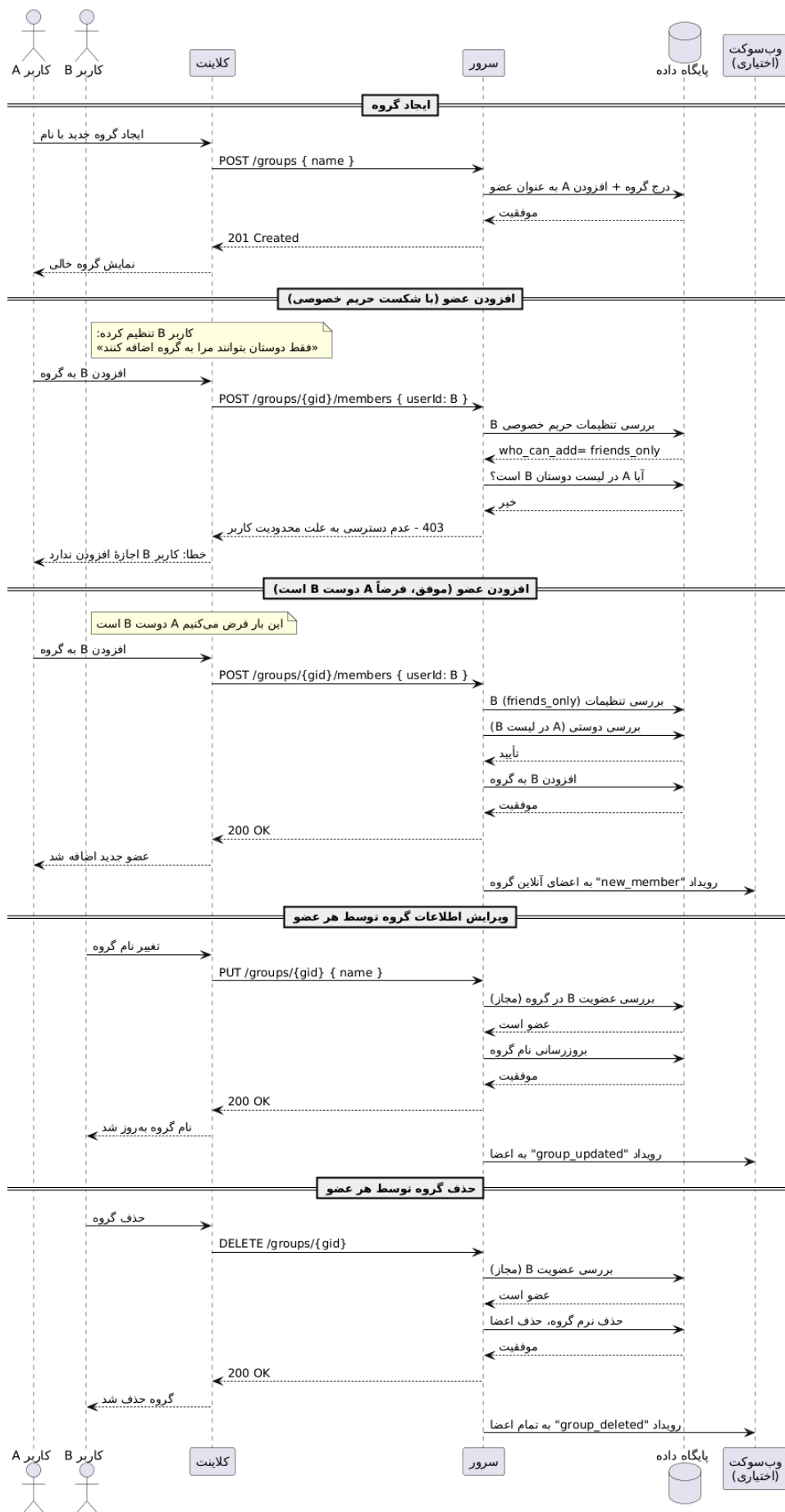
شکل ۴: نمودار Register Activity Diagram

نتیجه: با توجه به نمودار، تمام واریسی‌ها (یکتایی ایمیل، تطابق رمز عبور) در همان جریانی انجام می‌شود که کاربر نهایی ثبت می‌گردد. هیچ درخواست غیرهمگامی به یک سرویس کاربری جداگانه وجود ندارد؛ همه اعتبارسنجی‌ها و ذخیره‌سازی‌ها در همان پردازش (Process) اصلی صورت می‌گیرد.

۵-۱-۵ نمودار Group Lifecycle Sequence Diagram

این نمودار مواردی مانند افزودن عضو با رعایت تنظیمات حریم خصوصی و حذف گروه توسط هر عضو، و تعامل میان کلاینت، سرور و پایگاه داده را نشان می‌دهد. برای فهمیدن این‌که کدام بخش چه پیامی را به چه ترتیبی می‌فرستد، این نوع نمودار گزینه‌ای بسیار مناسب است. تصویر در صفحه بعد آورده شده است.

نتیجه: در طول تمام سناریوهای ایجاد گروه، افزودن عضو، ویرایش و حذف، تمام فراخوانی‌ها توسط سرور اصلی انجام می‌شود و پاسخ‌ها از همان‌جا برمی‌گردد. حتی رویداد WebSocket (برای اطلاع‌رسانی به اعضا) از داخل همان سرور صادر می‌شود؛ هیچ فراخوانی شبکه‌ای بین دو سرویس مجزا دیده نمی‌شود. این الگوی تعامل، یک هسته متمرکز را تأیید می‌کند که هم REST و هم اعلان‌های بلادرنگ را مدیریت می‌کند.



شکل ۵: نمودار Group Lifecycle Sequence Diagram

۵-۲ مقایسه رویکردهای موجود برای معماری سیستم

در معماری این برنامه، باید بین دو سطح مجزا تفاوت قائل شد:

۱. **معماری سیستم (System Architecture):** که به نحوه استقرار، تعداد پردازش‌ها، و مرزهای فیزیکی سرویس‌ها اشاره دارد (مثلاً انتخاب بین مونولیت و میکروسرویس). این سطح تعیین می‌کند که برنامه به چند قطعه‌ی مجزا برای اجرا (Deploy) تقسیم می‌شود.

۲. **معماری کد (Application/Code Architecture):** که به ساختار داخلی، لایه‌بندی، و نحوه‌ی چیدمان کدها در همان سرویس اشاره دارد (مانند معماری تمیز، لایه‌ای، یا DDD). این سطح تعیین می‌کند که کدها در یک سرویس چگونه از هم جدا شده و با یکدیگر تعامل دارند.

انتخاب مونولیت به این معنا نیست که کدها باید به هم ریخته و غیرقابل نگهداری باشند؛ بلکه می‌توان یک هسته‌ی ماژولار و لایه‌بندی‌شده درون یک مونولیت طراحی کرد. از سوی دیگر، انتخاب میکروسرویس نیز به خودی خود تضمین‌کننده‌ی کد تمیز نیست. بنابراین، ما در این پروژه، در سطح سیستم، معماری مونولیت ماژولار را برای کاهش پیچیدگی استقرار و عملیات (DevOps) انتخاب کرده‌ایم، اما در سطح کد، اصول معماری تمیز و لایه‌بندی را به صورت هدفمند و چابک (Agile) پیاده‌سازی می‌کنیم تا هم نگهداشت‌پذیری بلندمدت تضمین شود و هم از افت سرعت توسعه جلوگیری گردد.

۵-۳ راهکار ترکیبی (Hybrid)

معیار استفاده از اینترفیس: با وجود اینکه معماری تمیز بر استفاده‌ی گسترده از اینترفیس‌ها برای جداسازی لایه‌ها تأکید دارد، در این پروژه از رویکرد «اینترفیس به اندازه‌ی محدود» پیروی می‌کنیم. صرفاً برای ماژول‌هایی که دارای منطق پیچیده‌ی بیزینس با ریسک تغییر بالا هستند (مانند ماژول چت، جستجو و احراز هویت)، لایه‌ی اینترفیس (Repository Interface) تعریف می‌شود. اما برای ماژول‌های کاملاً CRUD (مثل پروفایل کاربر و اعلان‌ها)، مستقیماً از ORM جنگو در لایه‌ی Service استفاده می‌شود.

۴-۵ نمودار بسته (Package Diagram): تفکیک ماژول‌ها و لایه‌های پیاده‌سازی

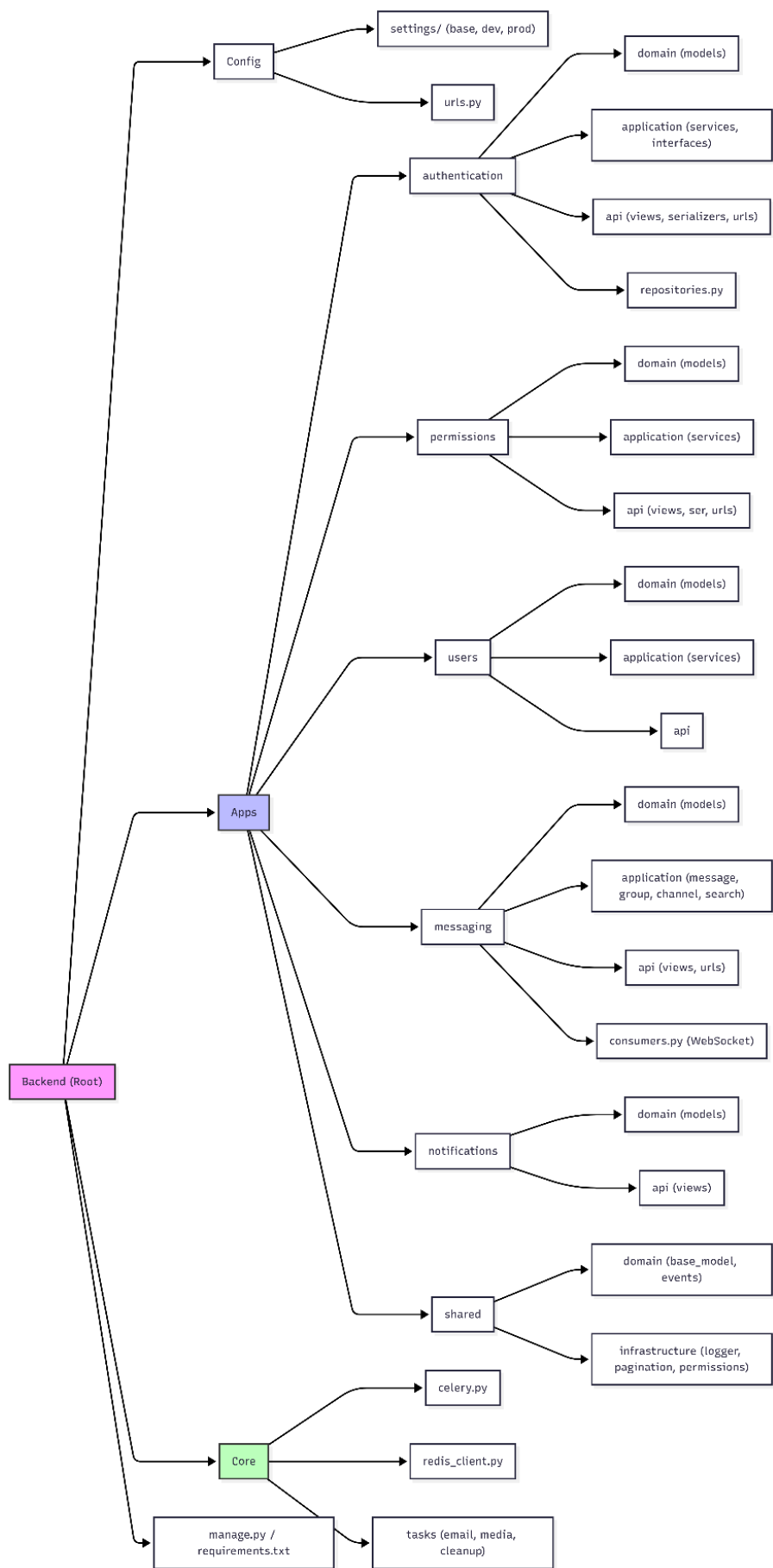
شرح نمودار و روند تصمیم‌گیری:

نمودار صفحه بعد، ساختار نهایی پوشه‌ها و ماژول‌های پروژه را نشان می‌دهد که حاصل چندین پرسش و پاسخ با ذی‌نفع (Stakeholder) پروژه است. در این مکالمات، بر دو نکته‌ی کلیدی تأکید شد:

۱. انتخاب معماری سیستم به‌صورت مونولیت (Monolith-First)، اما با تأکید بر اینکه این مونولیت باید درون خود دارای ماژول‌های مستقلی باشد که پیش‌تر به آن اشاره شد.

۲. در سطح معماری کد، توصیه شد که از اصول لایه‌بندی (Clean/Layered Architecture) الهام بگیریم، اما به‌صورت چابک (Agile) و با در نظر گرفتن محدودیت زمانی. به همین دلیل، در نمودار زیر مشاهده می‌شود که ماژول‌های حیاتی مانند messaging و authentication دارای لایه‌های مجزای domain، application و api هستند، درحالی‌که ماژول‌های ساده‌تر مانند notifications برای جلوگیری از ایجاد کدهای اضافی، تنها دو لایه‌ی domain و api را شامل می‌شوند.

در نهایت، برای کاهش Coupling بین ماژول‌ها، ارتباطات از طریق لایه‌های application/services و رویدادهای ناهمگام (Signals) طراحی شده است و کدهای مشترک نیز در پوشه‌ی shared متمرکز شده‌اند تا از تکرار کد جلوگیری شود.



شکل ۶: نمودار Package Diagram

۵-۵ رویکرد Monolith-First

با جداسازی منطقی بر اساس تحلیل نیازمندی‌ها، سیستم را به ۴ بخش منطقی تقسیم کردیم:

۱. بخش ۱: Gateway Layer

وظایف:

- مسیریابی درخواست‌ها
- محدودسازی نرخ (Rate Limiting)
- فشرده‌سازی و کشینگ (Caching)
- پایان‌دهی TLS

۲. بخش ۲: Base Core

این بخش، قلب تپنده برنامه است و خود از ماژول‌های مستقل درونی تشکیل می‌شود. نکته‌ی کلیدی در طراحی این بخش، ماژولار بودن در سطح کد و مستقل بودن وظایف است؛ به‌طوری‌که هر ماژول مسئولیت خاص خود را دارد و ارتباط بین آن‌ها از طریق لایه‌ی سرویس (و نه دسترسی مستقیم به مدل‌های یکدیگر) انجام می‌شود. با این حال، برای جلوگیری از مهندسی بیش‌ازحد (Over-Engineering)، جزئیات پیاده‌سازی لایه‌های داخلی مطابق با «بدهی فنی عمدی» که در بخش‌های قبلی به آن اشاره شد، مدیریت خواهد شد. در ادامه، وظایف هر یک از این ماژول‌های مستقل تشریح می‌شود:

(آ) ماژول احراز هویت:

- وظیفه: ثبت‌نام، ورود، خروج، صدور و اعتبارسنجی توکن، مدیریت 2FA، بازنشانی رمز عبور.

- خروجی: خطا یا User ID.

(ب) ماژول نقش‌ها:

- وظیفه: تعریف نقش‌های سراسری و ویژگی‌ها، تخصیص نقش به کاربر در یک کانال خاص، بررسی لحظه‌ای مجوزها (Permissions).
- خروجی: مجاز/غیرمجاز.

(ج) ماژول کاربران:

- وظیفه: پروفایل (ویرایش، مشاهده)، تنظیمات حریم خصوصی (چه کسی می‌تواند مرا به گروه اضافه کند)، لیست مخاطبین/دوستان.

• خروجی: متغیر.

(د) ماژول چت:

زیرماژول‌ها:

- Message Service: مسئول ارسال، ویرایش، حذف نرم و ذخیره‌سازی پیام‌ها (اعم از متن و رسانه) و تأمین تاریخچه چت‌ها برای کاربران است.
- Group Service: مسئول ایجاد و مدیریت گروه‌های ساده (افزودن عضو با رعایت تنظیمات حریم خصوصی، ویرایش و حذف گروه توسط هر عضو) است.
- Channel Service: مسئول ایجاد و مدیریت کانال‌ها (عمومی/خصوصی، نقش‌های کاربری سراسری، تاپیک‌ها و مجوزها) است.
- Search Service: مسئول جستجوی تمام‌متن سراسری در میان پیام‌های تمام چت‌هایی است که کاربر جاری به آن‌ها دسترسی دارد.

۳. بخش ۳: Real-Time Engine

وظیفه:

- نگهداری سوکت اتصال (Connection Socket)
- پخش رویدادها
- تشخیص حضور و اتمام حضور کاربر

خروجی: سوکت و وضعیت آن.

۴. بخش ۴: Background Worker

وظیفه:

- ارسال ایمیل
- پردازش رسانه‌ها (Media)
- پاک‌سازی فایل‌ها و توکن‌های بی‌استفاده

خروجی: متغیر.

۵-۶ توجیه فناوری‌ها

۵-۶-۱ Nginx

هدف این فناوری: پذیرش و هدایت هوشمندانه ترافیک ورودی، پایان‌دهی TLS و تحویل سریع فایل‌های ایستا است. از آنجا که یک برنامه پایتون نمی‌تواند به‌طور مؤثر محدودسازی نرخ (Rate Limit) را اعمال کند، فایل‌های استاتیک را با سرعت بالا ارائه دهد یا رمزنگاری HTTPS را به‌سبکی انجام دهد، از این فناوری استفاده کردیم.

هزینه: پیچیدگی کم و نگهداری آسان در بستر Docker.

۵-۶-۲ ASGI Server

مورد استفاده فناوری: اجرای تمام منطق سرور (REST و WebSocket) در یک قالب یکپارچه و بدون پراکندگی است. به‌طور پیش‌فرض، پایتون عمدتاً با پروتکل HTTP کار می‌کند، اما با استفاده از این فناوری می‌توان از اتصالات ناهمگام برای چت‌ها استفاده کرد. همچنین، مدل ASGI اساساً سریع‌تر است و می‌تواند هزاران اتصال سبک را با مصرف حافظه کم نگه دارد. اگر در آینده بخواهیم پروتکل جدیدی مانند HTTP/2 یا Server-Sent Events را اضافه کنیم، این بستر کاملاً آماده است.

هزینه: پیچیدگی در حد متوسط رو به پایین. یادگیری مفهوم «Consumer» به‌جای «View» تنها به چند ساعت زمان نیاز دارد. در مجموع، چون این فناوری درون خود اکوسیستم پایتون قرار دارد، پیچیدگی‌های معماری میکروسرویس را به همراه نخواهد داشت.

۵-۶-۳ PostgreSQL

هدف این فناوری: جهت ذخیره‌سازی مطمئن داده‌های رابطه‌ای و انجام جستجوی تمام‌متن (Full-Text Search) بدون نیاز به ابزار اضافی استفاده می‌شود. ما نیازمند جستجوی متنی سراسری روی پیام‌ها و ذخیره داده‌های رابطه‌ای پیچیده بودیم. گزینه‌های دیگر مانند MySQL قدرت جستجوی متنی کافی را نداشتند یا در نگهداری هم‌زمان اتصالات متعدد ضعیف عمل می‌کردند. علاوه بر این، پشتیبانی از نوع داده JSON بسیار مناسب است تا برای ذخیره مجوز نقش‌ها (Permissions) نیازی به جداول اضافی نداشته باشیم. همچنین، تراکشن‌های قوی این پایگاه داده (ACID) تضمین می‌کند که هیچ پیامی گم نشود یا به‌صورت نیمه‌کاره ثبت نگردد.

هزینه: پیچیدگی آن بیشتر از SQLite است، اما کاملاً استاندارد ارزیابی می‌شود. تنظیم بکاپ خودکار و نصب اولیه آن با Docker حدود ۱۰ دقیقه زمان می‌برد و از نظر منابع نیز تنها به ۱۰۰ الی ۲۰۰ مگابایت رم

نیاز دارد.

Celery ۴-۶-۵

هدف این فناوری: ایجاد یک صف پردازش برای کارهای پس‌زمینه و هماهنگ‌سازی عملیات زمان‌بر است. وقتی کاربر عکسی در چت آپلود می‌کند، تبدیل آن به Thumbnail یا ارسال ایمیل تأیید، فرایندی زمان‌بر است. اگر این کارها داخل همان پردازش (Thread) درخواست HTTP انجام می‌شد، کاربر چند ثانیه معطل می‌ماند و کل سرور کند می‌شد. همچنین می‌توان وظایف زمان‌بندی‌شده (Cron Job) مانند پاک‌سازی توکن‌های منقضی‌شده را به آن سپرد. در صورتی که پردازش رسانه‌ها سنگین شود، می‌توان تعداد Workerهای Celery را به‌طور جداگانه و بدون نیاز به ارتقای وب‌سرور افزایش داد.

هزینه: پیچیدگی در حد متوسط. نوشتن یک Task در این ابزار به‌سادگی نوشتن یک تابع معمولی پایتون همراه با یک Decorator است. تنها نیاز به اجرای یک پردازش جداگانه برای Worker داریم که این کار نیز توسط Docker به‌صورت خودکار مدیریت می‌شود.

Redis ۵-۶-۵

هدف این فناوری: جهت مدیریت واسط پیام‌ها، پردازش غیرهمگام کارهای سنگین و ایجاد یک حافظه سریع کشینگ (Caching) در خارج از چرخه درخواست کاربر استفاده می‌شود.

در این معماری با سه چالش اصلی مواجه بودیم:

۱. برای ارسال پیام زنده (Live) به کاربران آنلاین از طریق WebSocket، باید وضعیت آنلاین بودن کاربران را ردیابی می‌کردیم.

۲. ابزار Celery برای دریافت و مدیریت وظایف (مانند ارسال ایمیل) به یک واسط پیام (Message Broker) نیاز داشت.

۳. در صورت راه‌اندازی مجدد (Restart) برنامه، نشست (Session) کاربران و وظایف نیمه‌کاره نباید از بین می‌رفتند؛ لذا به یک حافظه سریع و مشترک نیاز بود.

علاوه بر این، می‌توان از Redis برای کش کردن نتایج درخواست‌های تکراری (مانند پروفایل کاربران) استفاده کرد تا بار پردازشی پایگاه داده کاهش یابد. قابلیت Pub/Sub در این فناوری باعث شد تا بتوانیم بدون نیاز به ابزارهای سنگین‌تری مانند RabbitMQ، سیگنال‌های داخلی را به‌خوبی مدیریت کنیم.

هزینه: نصب آن با Docker تنها با یک خط کد انجام می‌شود و نیازی به تعریف Schema ندارد. پیچیدگی عملیاتی آن بسیار پایین بوده و منابع مصرفی آن نیز ناچیز است (تنها چند مگابایت رم).

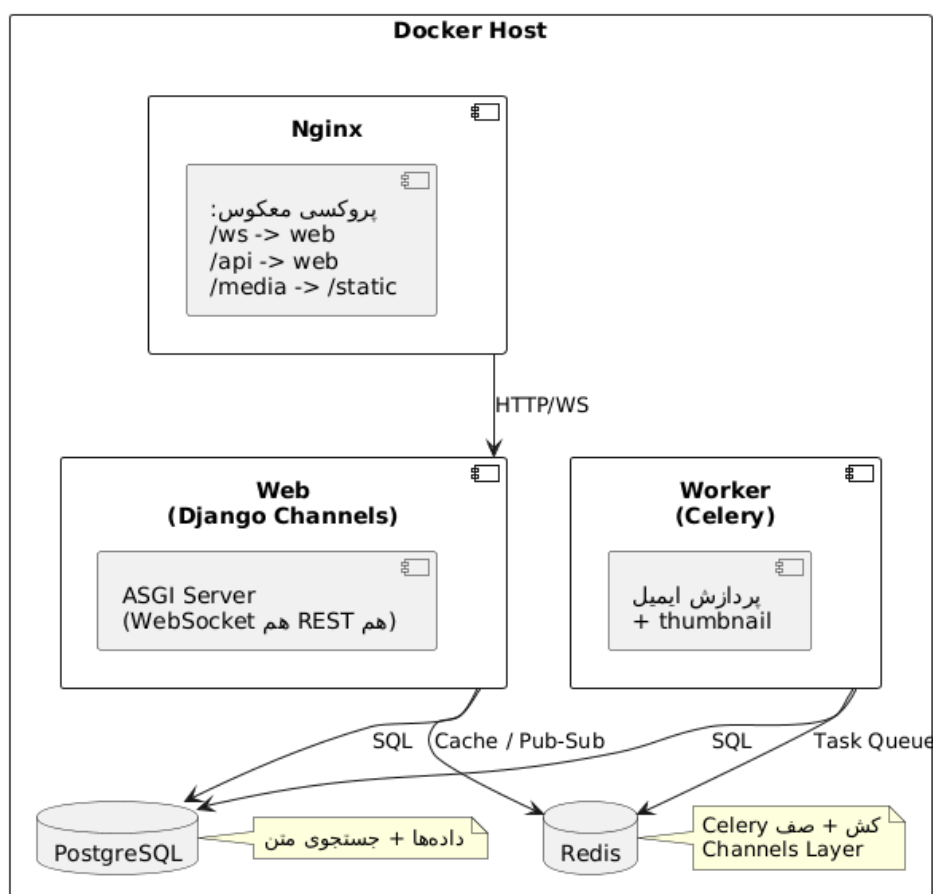
۵-۶-۶ Docker

هدف این فناوری: سیستم ما شامل ۵ بخش مختلف (Nginx، Web، Worker، Postgres، Redis) است که هر کدام به نصب و تنظیمات جداگانه‌ای نیاز دارند. با استفاده از Docker، پیکربندی کل سیستم به راحتی و به صورت یکپارچه انجام می‌شود. همچنین، تنها با اجرای دستور `docker-compose up` کل محیط سیستم یکجا راه‌اندازی می‌گردد. ایزوله بودن این کانتینرها به این معناست که به عنوان مثال، بروز مشکل در Redis هیچ تأثیر مخربی روی Postgres نخواهد گذاشت.

هزینه: نیاز به یادگیری نحوه نگارش Dockerfile و `docker-compose.yml` دارد؛ هرچند می‌توان با بهره‌گیری از مدل‌های زبانی بزرگ (LLMها) این هزینه زمانی را به شدت کاهش داد.

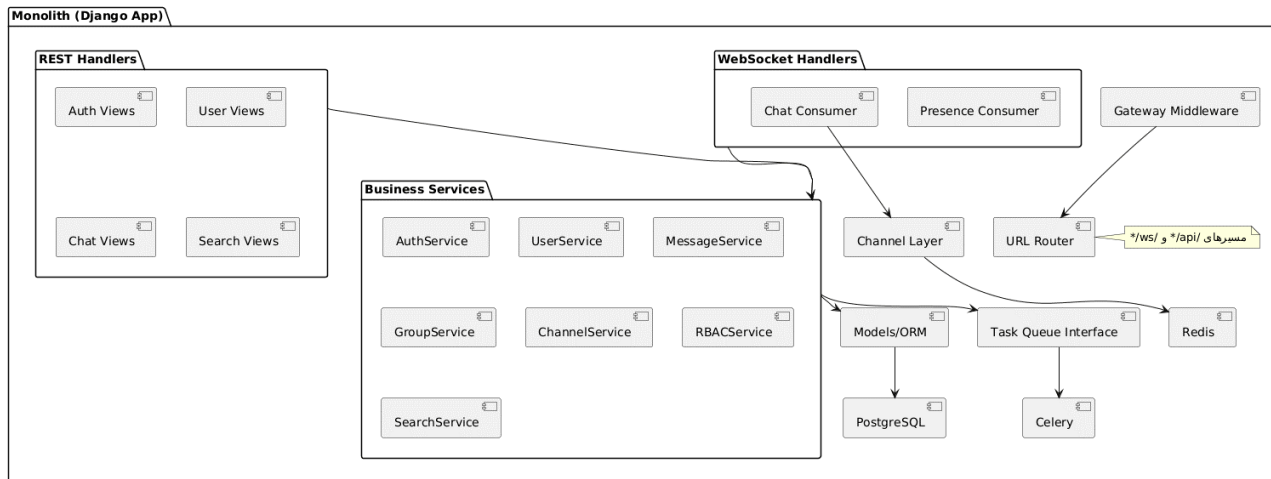
۵-۷ نمودار استقرار سرویس‌ها (Service Deployment Layout)

این نمودار نشان می‌دهد که اجزای مختلف سرور (وب، پردازشگر پس‌زمینه، پایگاه داده و کش) چگونه درون یک میزبان مشترک (Host) و تحت نظارت یک دروازه واحد (Nginx) قرار خواهند گرفت.



۵-۸ معماری اجزا (Component Architecture)

درون مرزهای برنامه، تمامی مسئولیت‌ها (شامل احراز هویت، چت، گروه، کانال، جستجو و مدیریت نقش‌ها) به صورت ماژول‌های مستقل اما هم‌پردازش (Co-process) طراحی شده‌اند و از طریق یک لایه مدل مشترک با پایگاه داده تعامل دارند. بر این اساس، توسعه و افزایش قابلیت‌های سیستم نیازی به تکه‌تکه کردن برنامه به سرویس‌های مجزا نخواهد داشت و پیچیدگی کار تنها در حد ماژولار بودن درون یک پردازش واحد باقی خواهد ماند.



۶ مدل سازی پایگاه داده (ERD)

در طراحی پایگاه داده این سامانه، هدف اصلی ایجاد ساختاری پایدار، بهینه و جلوگیری از افزونگی داده‌ها بوده است. به جای استفاده از روابط چندبه‌چند انتزاعی یا فیلدهای پویا (Polymorphic) که یکپارچگی داده‌ها را مخدوش می‌کنند، تمام روابط چندبه‌چند به جداول واسط فیزیکی تبدیل شده‌اند. همچنین با تفکیک داده‌های ساختاری از داده‌های ظاهری سنگین، سرعت واکنشی اطلاعات بهینه‌سازی شده است.

۱-۶ مدیریت کاربران و نمایه‌ها

اطلاعات حیاتی و امنیتی (مانند ایمیل و هاش پسورد) در جدول USER ذخیره می‌شود. برای سبک نگه‌داشتن این جدول و بهینه‌سازی کوئری‌های احراز هویت، اطلاعات ظاهری کاربر (مانند نام نمایشی، آواتار و بیو) در قالب یک رابطه ۱-به-۱ به جدول USER_PROFILE منتقل شده است. این تفکیک ساختاری باعث بالا رفتن سرعت عملیات‌های پایه سیستم می‌شود.

۲-۶ فضاهای کاری و گروه‌ها

کانال‌ها فضای انتشار یک‌طرفه هستند که می‌توانند چندین TOPIC داشته باشند. طبق بازخورد ذی‌نفع پروژه، فیلد Invite-Token به کانال اضافه شده تا مبنای ورود داوطلبانه کاربران از طریق لینک دعوت باشد. در سوی دیگر، گروه‌ها فضای گفتگوی چندنفره هستند که جدول GROUP_MEMBER وظیفه مدیریت اعضا و تعیین سطح دسترسی ادمین (Is_Admin) را بر عهده دارد و تعاملات دعوت نیز با جدول GROUP_INVITATION مدیریت می‌شود.

سیستم کنترل دسترسی کانال‌ها کاملاً پویا است؛ هر کانال نقش‌های خود را تعریف کرده و از طریق جدول واسط USER_ROLE به کاربران منتسب می‌کند. همچنین برای یکپارچه‌سازی و جلوگیری از ساخت جداول تکراری، یک جدول واحد به نام SHARED_PROFILE برای ذخیره آواتار و توضیحات کانال‌ها و گروه‌ها طراحی شده است. این جدول با دو کلید خارجی اختیاری (Nullable) به نام‌های Channel_ID و Group_ID کار می‌کند که به کمک یک شرط کنترلی (Check Constraint) در هر رکورد دقیقاً یکی از آن‌ها پر می‌شود.

۳-۶ هسته پیام‌رسانی و ارث‌بری جداول

جدول BASE_MESSAGE هسته مرکزی مدیریت پیام‌ها است. با توجه به اینکه پیام‌های آنی و زمان‌بندی شده بیش از ۹۰٪ در صفت‌ها مشترک هستند، از الگوی ارث‌بری در سطح جداول (Class Table Inheritance) استفاده شده است. این جدول صفت‌های فرستنده و متن را نگه می‌دارد. برای حل چالش مقصد پیام، سه کلید خارجی اختیاری (Topic_ID، Group_ID و Direct_Chat_ID) در آن قرار گرفته که در هر رکورد دقیقاً و فقط یکی از آن‌ها مقدار می‌گیرد.

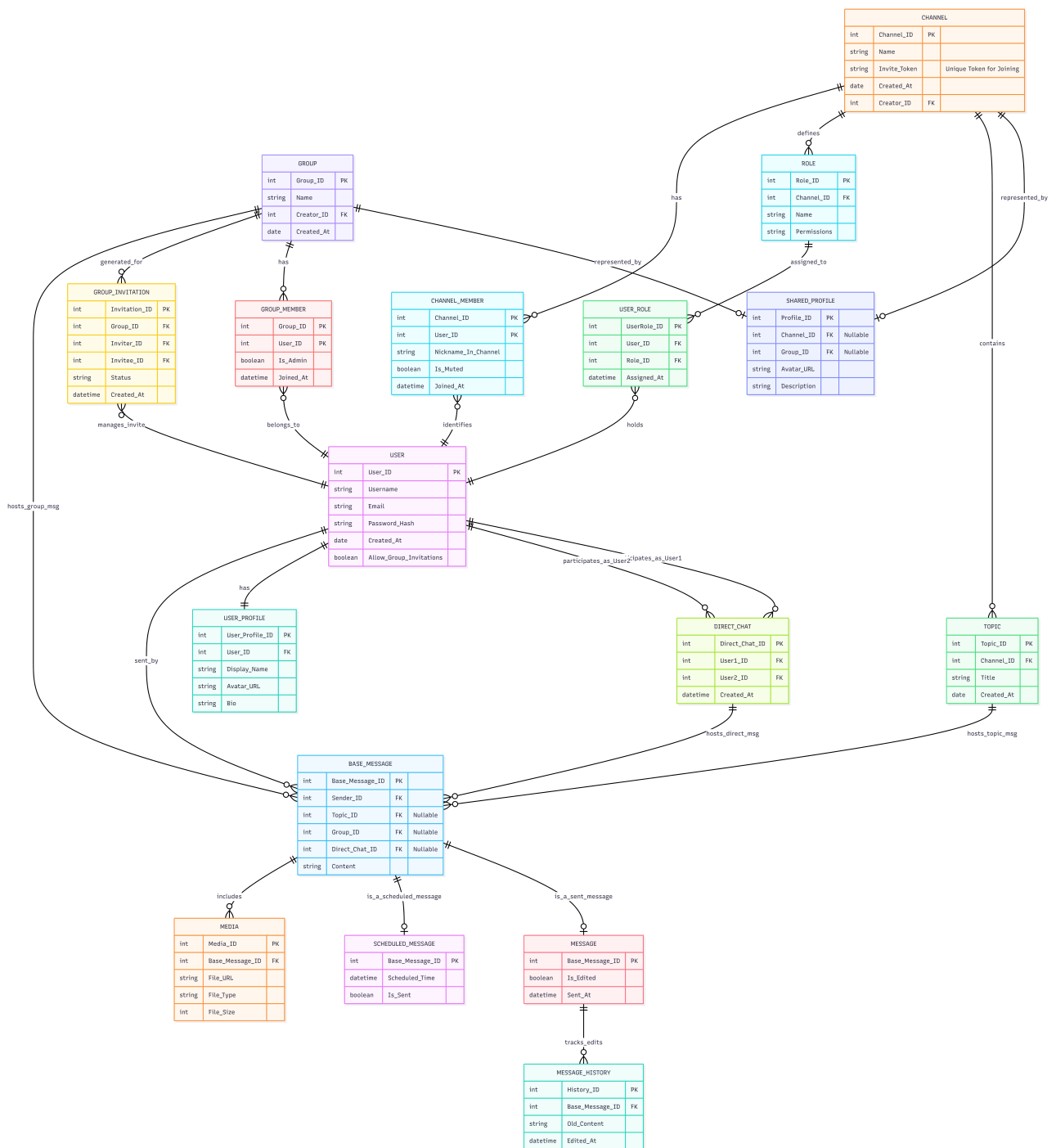
جدول MESSAGE و SCHEDULED_MESSAGE به عنوان فرزندان پیام پایه عمل می‌کنند. جدول MESSAGE صفت‌های پیام‌های live (مانند زمان ارسال و وضعیت ویرایش) را ذخیره می‌کند و SCHEDULED_MESSAGE وظیفه مدیریت صف پیام‌های آینده را بر عهده دارد. به لطف این الگوی ارث‌بری، جدول رسانه (MEDIA) مستقیماً به پیام پایه وصل می‌شود تا بدون پیچیدگی از پیوست فایل در هر دو نوع پیام پشتیبانی کند. جدول MESSAGE_HISTORY نیز تاریخچه ویرایش پیام‌ها را ردیابی می‌کند.

۴-۶ بهینه‌سازی چت‌های خصوصی

از آنجا که چت خصوصی ذاتاً همیشه بین دو کاربر شکل می‌گیرد، منطق متفاوتی برای آن در نظر گرفته شده است. برای حذف جدول واسط اضافی و کاهش تعداد الحاق‌ها (JOINها)، دو کلید خارجی مستقیم User1_ID و User2_ID درون خود جدول DIRECT_CHAT قرار گرفته‌اند که این کار کارایی واکشی گفتگوهای دو نفره را به شدت افزایش می‌دهد.

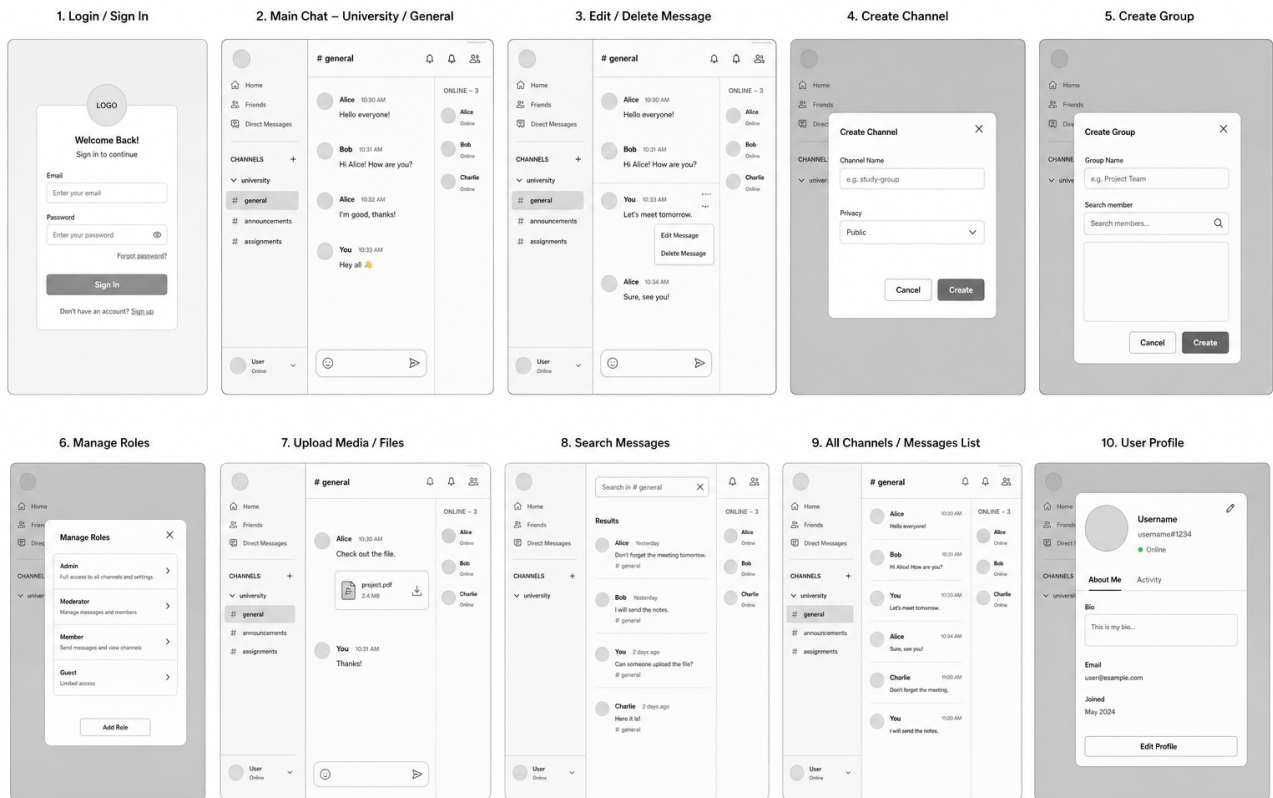
۵-۶ دستاوردهای کلیدی این معماری داده

- به جای استفاده از فیلدهای متنی غیراستاندارد برای مشخص کردن مقصد پیام یا نوع پروفایل، از کلیدهای خارجی صریح همراه با قید کنترلی استفاده شده که سرعت کوئری‌ها را به شدت بالا می‌برد.
- تجمیع منطق فرستنده، متن و رسانه‌ها در جدول پایه، فرآیند کانتینری‌سازی و توسعه کدهای بک‌اند را بسیار روان‌تر می‌کند.
- تفکیک داده‌های متنی سنگین (توضیحات و بیوگرافی‌ها) به جداول پروفایل مجزا، حجم رکوردهای پر تکرار دیتابیس را سبک نگه می‌دارد.



شکل ۷: نمودار erd

۷ طرح‌های رابط کاربری (Wireframes)



شکل ۸: روابط کاربری

۸ پیوست - API

۸-۱ بخش اول: قراردادهای API مدیریت هویت و نمایه (Identity & Profile Management)

۸-۱-۱ ثبت نام کاربر (Register User)

Endpoint: POST /api/auth/register/

نیاز به احراز هویت: خیر

Request Body - JSON

```
1 {  
2   "username": "nika_gh",  
3   "email": "nika@example.com",  
4   "password": "securepassword123"  
5 }
```

پاسخ (Response - 201 Created):

```
1 {  
2   "user_id": 1,  
3   "username": "nika_gh",  
4   "access_token": "eyJhbGciOiJIUzI1Ni...",  
5   "refresh_token": "def456..."  
6 }
```

قوانین تجاری (Business Rules): فیلد username باید کاملاً یکتا باشد. فیلد email نیز باید یکتا و معتبر باشد.

۸-۱-۲ ورود کاربر (Login User)

Endpoint: POST /api/auth/login/

نیاز به احراز هویت: خیر

Request Body - JSON


```

1 {
2   "username": "nika_gh",
3   "password": "securepassword123"
4 }

```

پاسخ (200 OK):

```

1 {
2   "access_token": "eyJhbGciOiJIUzI1Ni...",
3   "refresh_token": "def456..."
4 }

```

قوانین تجاری (Business Rules): اطلاعات هویتی (Credentials) را اعتبارسنجی می‌کند و توکن‌های نشست فعال (Active Session Tokens) را برمی‌گرداند.

۳-۱-۸ خروج کاربر (Logout User)

POST /api/auth/logout/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

:Request Body - JSON

```

1 {
2   "refresh_token": "def456..."
3 }

```

پاسخ (200 OK):

```

1 {
2   "message": "Successfully logged out."
3 }

```

قوانین تجاری (Business Rules): توکن آپدیت (Refresh Token) ارائه‌شده را در پایگاه داده بک‌اند در Blacklist قرار می‌دهد تا نشست کاربر باطل (Invalidate) شود.

۴-۱-۸ درخواست بازیابی رمز عبور (Password Reset Request)

POST /api/auth/password-reset/ :Endpoint

نیاز به احراز هویت : خیر

:Request Body - JSON

```
1 {  
2   "email": "nika@example.com"  
3 }
```

پاسخ (200 OK):

```
1 {  
2   "message": "If an account exists, a reset link has been sent."  
3 }
```

قوانین تجاری (Business Rules): به منظور جلوگیری از حملات User Enumeration، در پاسخ مشخص نمی‌کند که آیا ایمیل در پایگاه داده وجود دارد یا خیر.

۵-۱-۸ دریافت نمایه (Get My Profile)

GET /api/users/me/profile/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

:Request Body ندارد

پاسخ (200 OK):

```
1 {  
2   "user_id": 1,  
3   "username": "nika_gh",  
4   "display_name": "Nika Ghaderi",  
5   "avatar_url": "https://storage/avatars/nika.jpg",  
6   "bio": "Backend Developer",  
7   "allow_group_invitations": true  
8 }
```

قوانین تجاری (Business Rules): نمایه کاربری مرتبط با توکن دسترسی (Access Token) اطلاعات ارائه شده را بازیابی می کند.

۶-۱-۸ بروزرسانی نمایه (Update My Profile)

Endpoint: PATCH /api/users/me/profile/

نیاز به احراز هویت: بله (Bearer Token)

Request Body - JSON

```
1 {
2   "display_name": "Nika",
3   "bio": "Updating my bio for the sprint!",
4   "allow_group_invitations": false
5 }
```

پاسخ (200 OK):

```
1 {
2   "user_id": 1,
3   "username": "nika_gh",
4   "display_name": "Nika",
5   "avatar_url": "https://storage/avatars/nika.jpg",
6   "bio": "Updating my bio for the sprint!",
7   "allow_group_invitations": false
8 }
```

قوانین تجاری (Business Rules): بروزرسانی جزئی (Partial Update) انجام می شود؛ بدین معنا که تنها فیلدهای ارائه شده تغییر می کنند. بارگذاری آواتار به صورت جداگانه و از طریق درخواست multipart/form-data مدیریت می شود.

۷-۱-۸ مشاهده نمایه کاربری دیگران (View Another User's Profile)

Endpoint: GET /api/users/{username}/profile/

نیاز به احراز هویت: بله (Bearer Token)

بدنه درخواست (Request Body): ندارد

پاسخ (200 OK):

```
1 {  
2   "user_id": 2,  
3   "username": "samyar_l",  
4   "display_name": "Samyar Lajevardi",  
5   "avatar_url": "https://storage/avatars/samyar.jpg",  
6   "bio": "Product Owner"  
7 }
```

قوانین تجاری (Business Rules): اطلاعات عمومی نمایه را برمی گرداند و تنظیمات خصوصی مانند allow_group_invitations را شامل نمی شود. در صورتی که نام کاربری وجود نداشته باشد، خطای 404 Not Found برمی گرداند.

۸-۲ بخش دوم: قراردادهای API فضاهاى کارى (کانال‌ها، تاپیک‌ها و نقش‌ها) (Workspaces)

۸-۲-۱ دریافت کانال‌ها (Fetch My Channels)

Endpoint: GET /api/channels/

نیاز به احراز هویت : بله (Bearer Token)

پاسخ (200 OK): آرایه‌ای از اشیاء کانال (Channel Objects) را برمی‌گرداند که کاربر درخواست‌دهنده در حال حاضر عضو آنهاست.

قوانین تجارى (Business Rules): برای پر کردن نوار کناری (Sidebar) رابط کاربری فرانت‌اند پس از ورود کاربر ضرورى است. رکوردها را با Join زدن جداول CHANNEL_MEMBER و CHANNEL برای کاربر احراز هویت شده (Authenticated User) استخراج می‌کند.

۸-۲-۲ ایجاد کانال (Create a Channel)

Endpoint: POST /api/channels/

نیاز به احراز هویت : بله (Bearer Token)

Request Body - JSON

```
1 {  
2   "name": "Sharif SWE Project"  
3 }
```

پاسخ (201 Created):

```
1 {  
2   "channel_id": 1,  
3   "name": "Sharif SWE Project",  
4   "creator_id": 15,  
5   "default_topic_id": 1,  
6   "created_at": "2026-06-19T14:30:00Z"  
7 }
```

قوانین تجارى (Business Rules): به کاربری که کانال را ایجاد می‌کند، به‌طور پیش‌فرض نقش مالک

(Owner) داده می‌شود. همچنین به‌طور خودکار هنگام ایجاد کانال، یک تاپیک پیش‌فرض با نام general می‌سازد.

۳-۲-۸ ویرایش/حذف کانال (Edit/Delete Channel)

PATCH /api/channels/{channel_id}/ :Endpoint

DELETE /api/channels/{channel_id}/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

:Request Body - JSON for PATCH

```
1 {  
2   "name": "Updated Server Name"  
3 }
```

پاسخ (200 OK for PATCH, 204 No Content for DELETE):

```
1 {  
2   "channel_id": 1,  
3   "name": "Updated Server Name"  
4 }
```

قوانین تجاری (Business Rules): کاربر درخواست‌دهنده باید دارای دسترسی "MANAGE_CHANNEL" بوده یا خود ایجادکننده اصلی (Creator_ID) باشد. حذف یک کانال باعث حذف آبشاری (Cascade Delete) تمامی تاپیک‌ها، نقش‌ها و عضویت‌های مرتبط با آن می‌شود.

۴-۲-۸ پیوستن به کانال (Join a Channel)

POST /api/channels/{channel_id}/join/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

:Request Body - JSON Optional

```
1 {  
2   "nickname_in_channel": "Sprint Master"  
3 }
```

پاسخ (201 Created):

```

1 {
2   "channel_id": 1,
3   "user_id": 42,
4   "nickname_in_channel": "Sprint Master Nika",
5   "joined_at": "2026-06-19T14:45:00Z"
6 }

```

قوانین تجاری (Business Rules): یک رکورد در جدول CHANNEL_MEMBER ایجاد می‌کند.

فیلد nickname_in_channel اختیاری است؛ در صورت ارسال نشدن، بک‌اند به‌طور پیش‌فرض از display_name سراسری کاربر استفاده می‌کند. پس از پیوستن، دسترسی‌های نقش پیش‌فرض @everyone به کاربر داده می‌شود.

۵-۲-۸ ایجاد نقش سفارشی (Create a Custom Role)

POST /api/channels/{channel_id}/roles/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

:Request Body - JSON

```

1 {
2   "name": "Moderator",
3   "permissions": ["DELETE_MESSAGES", "MANAGE_TOPICS", "KICK_MEMBERS", "
4     SEND_MEDIA"]
5 }

```

پاسخ (201 Created):

```

1 {
2   "role_id": 3,
3   "channel_id": 1,
4   "name": "Moderator",
5   "permissions": ["DELETE_MESSAGES", "MANAGE_TOPICS", "KICK_MEMBERS"]
6 }

```

قوانین تجاری (Business Rules): کاربر درخواست‌دهنده باید دسترسی "MANAGE_ROLES" را داشته باشد. اعتبارسنجی می‌کند که دسترسی‌های درخواستی، زیرمجموعه معتبری از قابلیت‌های سیستم باشند.

۶-۲-۸ ترک کانال (Leave a Channel)

Endpoint: DELETE /api/channels/{channel_id}/leave/

نیاز به احراز هویت: بله (Bearer Token)

Request Body: ندارد

پاسخ (204 No Content): بدنه خالی.

قوانین تجاری (Business Rules): رکورد کاربر درخواست‌دهنده را از جدول CHANNEL_MEMBER برای این کانال خاص حذف می‌کند. همچنین هرگونه نقش سفارشی کاربر در این کانال را به‌صورت آبشاری از جدول USER_ROLE پاک می‌کند.

۷-۲-۸ ویرایش/حذف نقش سفارشی (Edit/Delete Custom Role)

Endpoint: PATCH /api/channels/{channel_id}/roles/{role_id}/

Endpoint: DELETE /api/channels/{channel_id}/roles/{role_id}/

نیاز به احراز هویت: بله (Bearer Token)

Request Body - JSON for PATCH

```
1 {  
2   "permissions": ["DELETE_MESSAGES", "MANAGE_TOPICS"]  
3 }
```

پاسخ (200 OK): شیء نقش بروزرسانی شده را برمی‌گرداند.

قوانین تجاری (Business Rules): سلسله‌مراتب دسترسی‌ها (Permission Hierarchy) را اعتبارسنجی می‌کند. نقش مالک (Owner) قابل حذف نیست.

۸-۲-۸ تخصیص نقش به عضو (Assign Role to Member)

Endpoint: POST /api/channels/{channel_id}/members/{user_id}/roles/

نیاز به احراز هویت: بله (Bearer Token)

Request Body - JSON


```

1 {
2   "role_id": 3
3 }

```

پاسخ (201 Created):

```

1 {
2   "userrole_id": 12,
3   "user_id": 42,
4   "role_id": 3,
5   "assigned_at": "2026-06-19T15:00:00Z"
6 }

```

قوانین تجاری (Business Rules): کاربر را به یک نقش سفارشی در جدول واسطه USER_ROLE متصل می‌کند. کاربر درخواست‌دهنده باید دسترسی "MANAGE_ROLES" را داشته باشد.

۹-۲-۸ ایجاد تاپیک (Create a Topic)

POST /api/channels/{channel_id}/topics/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

:Request Body - JSON

```

1 {
2   "title": "general"
3 }

```

پاسخ (201 Created):

```

1 {
2   "topic_id": 5,
3   "channel_id": 1,
4   "title": "general",
5   "created_at": "2026-06-19T15:10:00Z"
6 }

```

قوانین تجاری (Business Rules): کاربر باید دارای نقشی با دسترسی "MANAGE_TOPICS" باشد.

۸-۲-۱۰ حذف تاپیک (Delete a Topic)

Endpoint: DELETE /api/channels/{channel_id}/topics/{topic_id}/

نیاز به احراز هویت : بله (Bearer Token)

Request Body: ندارد

پاسخ (204 No Content): بدنه خالی.

قوانین تجاری (Business Rules): کاربر درخواست دهنده باید دسترسی "MANAGE_TOPICS" را داشته باشد. بک اند اعتبارسنجی می کند که کانال پیش از اجازه حذف، حداقل یک تاپیک فعال دیگر داشته باشد تا سرور در یک وضعیت خالی و غیرقابل چت (Empty un-chatable state) گیر نکند. حذف یک تاپیک باعث حذف آبشاری (Cascade Delete) تمام پیام های (BASE_MESSAGE) مرتبط با این topic_id می شود.

۳-۸ بخش سوم: قراردادهای API فضاهای خصوصی (گروه‌ها و چت‌های مستقیم) (Private Spaces)

۱-۳-۸ دریافت گروه‌ها و چت‌های مستقیم (Fetch My Groups & DMs)

GET /api/groups/ :Endpoint

GET /api/dms/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

پاسخ (200 OK): آرایه‌ای از اشیاء گروه (Group Objects) و اشیاء چت مستقیم (Direct Chat Objects) را برمی‌گرداند که کاربر در آن‌ها عضویت دارد. قوانین تجاری (Business Rules): رابط کاربری پیام‌های خصوصی را پر می‌کند. همچنین عکس پروفایل‌ها باید نمایش داده شوند.

۲-۳-۸ ایجاد چت مستقیم (Create a Direct Chat)

POST /api/dms/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

:Request Body - JSON

```
1 {  
2   "target_user_id": 42  
3 }
```

پاسخ (201 Created):

```
1 {  
2   "direct_chat_id": 8,  
3   "user1_id": 1,  
4   "user2_id": 42,  
5   "created_at": "2026-06-19T16:00:00Z"  
6 }
```

قوانین تجاری (Business Rules): بک‌اند ابتدا باید بررسی کند که آیا از قبل یک چت مستقیم (DIRECT_CHAT) میان این دو کاربر وجود دارد یا خیر. در صورت وجود، به‌جای ایجاد چت تکراری،

شناسه چت فعلی را با کد وضعیت OK 200 برمی گرداند.

۳-۳-۸ حذف چت مستقیم (Delete a Direct Chat)

DELETE /api/dms/{dm_id}/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

Request Body : ندارد

پاسخ (204 No Content): بدنه خالی.

قوانین تجاری (Business Rules): کاربر درخواست دهنده باید User1_ID یا User2_ID باشد. این یک حذف فیزیکی و سراسری (Global Hard Delete) است؛ بدین معنا که تمامی پیام های (BASE_MESSAGE) مرتبط با این dm_id برای هر دو کاربر به صورت آشناری از بین می روند.

۴-۳-۸ ایجاد گروه (Create a Group)

POST /api/groups/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

:Request Body - JSON

```
1 {
2   "name": "Weekend CTF Team"
3 }
```

پاسخ (201 Created):

```
1 {
2   "group_id": 3,
3   "name": "Weekend CTF Team",
4   "creator_id": 1,
5   "created_at": "2026-06-19T16:15:00Z"
6 }
```

قوانین تجاری (Business Rules): به طور خودکار کاربر درخواست دهنده را با مقدار true برای فیلد Is_Admin در جدول GROUP_MEMBER درج می کند.

۵-۳-۸ ویرایش اطلاعات گروه (Edit Group Details)

PATCH /api/groups/{group_id}/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

:Request Body - JSON

```
1 {  
2   "name": "Finals Study Group"  
3 }
```

پاسخ (200 OK): شیء گروه بروزرسانی شده را برمی گرداند.

قوانین تجاری (Business Rules): هر یک از اعضای گروه (فارغ از وضعیت مدیریت) صلاحیت ویرایش اطلاعات گروه را دارد. بک اند تنها باید تأیید کند که کاربر در جدول GROUP_MEMBER وجود دارد.

۶-۳-۸ حذف یا ترک گروه (Delete or Leave a Group)

DELETE /api/groups/{group_id}/ :Endpoint

DELETE /api/groups/{group_id}/leave/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

Request Body : ندارد

پاسخ (204 No Content): بدنه خالی.

قوانین تجاری (Business Rules): هر یک از اعضای گروه (فارغ از وضعیت مدیریت) صلاحیت اجرای حذف فیزیکی و سراسری (Global Hard Delete) گروه و تمامی پیام های آن را دارد.

۷-۳-۸ ارسال دعوت نامه گروه (Send Group Invitation)

POST /api/groups/{group_id}/invitations/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

:Request Body - JSON

```
1 {  
2   "invitee_id": 99  
3 }
```

پاسخ (201 Created):

```
1 {
2   "invitation_id": 45,
3   "group_id": 3,
4   "inviter_id": 1,
5   "invitee_id": 99,
6   "status": "PENDING"
7 }
```

قوانین تجاری (Business Rules): بک‌اند باید جدول USER مربوط به کاربر هدف را کوئری بگیرد.

۸-۳-۸ پاسخ به دعوت‌نامه گروه (Respond to Group Invitation)

Endpoint: PATCH /api/invitations/{invitation_id}/

نیاز به احراز هویت: بله (Bearer Token)

Request Body - JSON

```
1 {
2   "status": "ACCEPTED"
3 }
```

پاسخ (200 OK):

```
1 {
2   "invitation_id": 45,
3   "status": "ACCEPTED"
4 }
```

قوانین تجاری (Business Rules): مقادیر "ACCEPTED" یا "DECLINED" را می‌پذیرد. در صورت پذیرش، بک‌اند تراکنشی را اجرا می‌کند که ضمن بروزرسانی وضعیت دعوت‌نامه، به‌طور همزمان یک سطر جدید برای کاربر دعوت‌شده در جدول GROUP_MEMBER درج می‌نماید.

۸-۴ بخش چهارم: قراردادهای API هسته پیام‌رسانی (The Messaging Core)

۸-۴-۱ ارسال پیام متنی (Send a Message - Text)

Endpoint: POST /api/messages/

نیاز به احراز هویت: بله (Bearer Token)

Request Body - JSON

```
1 {  
2   "topic_id": 5,  
3   "group_id": null,  
4   "direct_chat_id": null,  
5   "content": "Does anyone have the link to the sprint backlog?"  
6 }
```

پاسخ (201 Created):

```
1 {  
2   "base_message_id": 1024,  
3   "sender_id": 1,  
4   "content": "Does anyone have the link to the sprint backlog?",  
5   "sent_at": "2026-06-19T17:00:00Z",  
6   "is_edited": false  
7 }
```

قوانین تجاری (Business Rules): فرانت‌اند باید دقیقاً یک شناسه هدف (topic_id, group_id یا direct_chat_id) را مشخص کرده و بقیه را null بگذارد. بک‌اند پیش از درج سطر در جداول BASE_MESSAGE و MESSAGE، اعتبارسنجی می‌کند که کاربر درخواست‌دهنده عضو فعالی از فضای هدف باشد.

۸-۴-۲ پیوست رسانه به پیام (Attach Media to a Message)

Endpoint: POST /api/messages/{base_message_id}/media/

نیاز به احراز هویت: بله (Bearer Token)

Request Body: multipart/form-data (بارگذاری فایل)

پاسخ (201 Created):

```
1 {
2   "media_id": 42,
3   "base_message_id": 1024,
4   "file_url": "https://storage/attachments/sprint_backlog.pdf",
5   "file_type": "application/pdf",
6   "file_size": 2048000
7 }
```

قوانین تجاری (Business Rules): کاربر درخواست دهنده باید همان فرستنده اصلی پیام (Sender_ID) باشد. بک اند محدودیت های حداکثر اندازه فایل و انواع مجاز آن را بر اساس تصمیمات مهندسی تیم بررسی می کند. نکته بسیار مهم این است که اگر پیام در یک کانال ارسال می شود، بک اند پیش از پذیرش فایل و درج رکورد در جدول MEDIA، باید تأیید کند که کاربر دارای نقشی با مجوز "SEND_MEDIA" است.

۳-۴-۸ دریافت تاریخچه پیام ها (Fetch Message History)

GET /api/messages/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

پارامترهای کوئری (Query Parameters): ?topic_id=5&limit=50&offset=0

پاسخ (200 OK):

```
1 {
2   "count": 150,
3   "results": [
4     {
5       "base_message_id": 1024,
6       "sender_id": 1,
7       "content": "Does anyone have the link to the sprint backlog?",
8       "is_edited": false,
9       "sent_at": "2026-06-19T17:00:00Z",
10      "media": [
11        {
12          "file_url": "https://storage/attachments/sprint_backlog.pdf",
```



```

13         "file_type": "application/pdf"
14     }
15 ]
16 }
17 ]
18 }

```

قوانین تجاری (Business Rules): دسترسی کاربر به فضای درخواستی را اعتبارسنجی می‌کند. از صفحه‌بندی (Pagination شامل limit و offset) استفاده می‌کند تا از اعمال بار اضافی به رابط کاربری فرانت‌اند و پایگاه داده در حین بارگذاری اولیه جلوگیری شود.

۴-۴-۸ ویرایش پیام (Edit a Message)

PATCH /api/messages/{base_message_id}/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

:Request Body - JSON

```

1 {
2     "content": "Found the link, nevermind!"
3 }

```

پاسخ (200 OK): شیء پیام بروزرسانی شده را برمی‌گرداند.

قوانین تجاری (Business Rules): کاربر درخواست‌دهنده باید فرستنده پیام (Sender_ID) باشد. بک‌اند یک تراکنش (Transaction) اجرا می‌کند: محتوای قدیمی را در جدول MESSAGE_HISTORY درج می‌کند، محتوای جدید BASE_MESSAGE را بروزرسانی کرده و در نهایت وضعیت Is_Edited در جدول MESSAGE را روی true تنظیم می‌کند.

۵-۴-۸ حذف پیام (Delete a Message)

DELETE /api/messages/{base_message_id}/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

:Request Body ندارد

پاسخ (204 No Content): بدنه خالی.

قوانین تجاری (Business Rules): طبق نیازمندی‌های صریح پروژه، این یک حذف سراسری (Global Delete) است و سطر را به‌طور دائم برای همه کاربران از پایگاه داده پاک می‌کند. کاربر درخواست‌دهنده باید فرستنده پیام (Sender_ID) بوده یا مدیری با مجوز "DELETE_MESSAGES" در فضای مربوطه باشد.

۶-۴-۸ جستجوی متنی (Text Search)

Endpoint: GET /api/messages/search/

نیاز به احراز هویت: بله (Bearer Token)

پارامترهای کوئری (Query Parameters): ?q=sprint&group_id=3

پاسخ (200 OK): یک لیست صفحه‌بندی‌شده از اشیاء پیام که با عبارت جستجو مطابقت دارند را برمی‌گرداند.

قوانین تجاری (Business Rules): بر اساس توضیحات هماهنگ‌شده با TA، این جستجو صرفاً بر روی فیلد Content از جدول BASE_MESSAGE اجرا می‌شود. سیستم در نام فایل‌ها، نام‌های کاربری یا بیوگرافی کاربران جستجو نمی‌کند. دامنه جستجو اکیداً به فضاهایی محدود است که کاربر مجاز به دسترسی به آن‌ها باشد.

۵-۸ بخش پنجم: قراردادهای API ویژگی‌های پیشرفته و ارتباطات بلادرنگ

۱-۵-۸ اتصال وب‌سوکت (چت live و اعلانات) (WebSocket Connection)

Endpoint: ws://{domain}/ws/stream/

نیاز به احراز هویت: بله (ارسال توکن در URL اتصال یا Payload اولیه)

Payload Structure (Incoming to Client)

```
1 {  
2   "event_type": "NEW_MESSAGE",  
3   "data": {  
4     "message_id": 1025,  
5     "content": "Hey team, looking at the backlog now."  
6   }  
7 }
```

قوانین تجاری (Business Rules): سرور رویدادهای live (نظیر NEW_MESSAGE، MESSAGE_DELETED و NEW_NOTIFICATION) را از طریق این اتصال پایدار (Persistent Connection) به کلاینت ارسال (Push) می‌کند. نیازی به آپدیت دستی (F5 Refresh) توسط کاربر نیست.

۲-۵-۸ دریافت اعلانات از دست‌رفته (Fetch Missed Notifications)

Endpoint: GET /api/notifications/

نیاز به احراز هویت: بله (Bearer Token)

پاسخ (200 OK):

```
1 [  
2   {  
3     "notification_id": 88,  
4     "type": "GROUP_INVITE",  
5     "content": "Samyar invited you to 'CTF Team'",  
6     "is_read": false,  
7     "created_at": "2026-06-19T18:00:00Z"  
8   }  
9 ]
```

قوانین تجاری (Business Rules): اعلانات باید بر ای کاربران آفلاین ذخیره شوند تا هشدارهای از دست رفته در زمان قطعی اتصال وبسوکت را بازیابی (Pull) کند.

۳-۵-۸ زمان بندی پیام (Schedule a Message)

POST /api/messages/scheduled/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

:Request Body - JSON

```
1 {
2   "topic_id": 5,
3   "content": "Reminder: Daily Standup in 10 mins!",
4   "scheduled_time": "2026-06-20T09:00:00Z"
5 }
```

پاسخ (201 Created):

```
1 {
2   "scheduled_id": 12,
3   "status": "QUEUED"
4 }
```

قوانین تجاری (Business Rules): پیام در جدول SCHEDULED_MESSAGE ذخیره می شود. یک صف وظایف پس زمینه (Background Task Queue) (مانند Celery یا RabbitMQ) وظیفه دارد آن را دقیقاً در زمان مقرر (scheduled_time) به جدول BASE_MESSAGE منتقل کند، فارغ از اینکه کلاینت کاربر در آن لحظه آنلاین باشد یا خیر.

۴-۵-۸ لغو پیام زمان بندی شده (Cancel a Scheduled Message)

DELETE /api/messages/scheduled/{scheduled_id}/ :Endpoint

نیاز به احراز هویت : بله (Bearer Token)

پاسخ (204 No Content): بدنه خالی.

قوانین تجاری (Business Rules): به کاربر اجازه می دهد یک پیام زمان بندی شده را پیش از اجرای آن توسط پردازشگر پس زمینه (Background Worker) لغو کند. این درخواست باید توسط فرستنده اصلی پیام (Sender_ID) ارسال شود.